

中图分类号: TN47

论文编号: 10357P18301191



专业硕士学位论文

基于程序中数据流访存模式的硬件预取技术的研究

作者姓名	汪杰
专业学位类别	工程硕士
专业学位领域	集成电路工程
指导教师	杨菲 卢文娟
企业导师	颜世云

Research on Hardware Prefetch Technology Based on the Stream Access Pattern in the Program

A Dissertation Submitted for the Degree of Master

Candidate: WANG Jie

Supervisor: YANG Fei Lu Wenjuan

Corporate Supervisor: YAN Shiyun

中图分类号: TN47

论文编号: 10357P18301191

硕 士 学 位 论 文

基于程序中数据流访存模式的硬件预取技术的研究

作者姓名	汪杰	申请学位级别	工程硕士
指导教师姓名	杨菲 卢文娟	职 称	副教授 讲师
企业导师姓名	颜世云	职 称	高级工程师
专业名称	集成电路工程	研究方向	数字集成电路设计
学习时间自	2018 年 09 月 01 日	起至	2021 年 06 月 30 日止
论文提交日期	2021 年 04 月 15 日	论文答辩日期	2021 年 5 月 23 日

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得安徽大学或其他教育机构的学位或证书而使用过的材料。如有学术不端行为，一切后果由本人承担，与导师和安徽大学无关。

学位论文作者签名：

汪杰

签字日期：

2021 年 5 月 21 日

学位论文版权使用授权书

本学位论文作者完全了解安徽大学有关保留、使用学位论文的规定，有权保留并向国家有关部门或机构送交论文的复印件和磁盘，允许论文被查阅和借阅。本人授权安徽大学可以将学位论文的全部内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

(保密的学位论文在解密后适用本授权书)

学位论文作者签名：

汪杰

导师签名：

杨菲

签字日期：

2021 年 5 月 21 日

签字日期：

2021 年 5 月 21 日

摘 要

自 20 世纪 70 年代以来，处理器的研发趋势始终关注如何提高内核中指令的执行效率，而主存储器却主要聚焦于存储容量的增大，忽略了速度的提升。处理器与主存发展趋势的不同，造成了两者之间访存速度难以匹配，直接导致了影响计算机性能的“存储墙”问题。为了试图弥合处理器与主存之间的速度差，计算机架构师们普遍采用在系统中插入多级缓存的层次型存储结构。然而，缓存的容量与主存相比毕竟有限，访存期间一旦出现缓存缺失，就会带来很大的不命中处罚延时，从而造成处理器访存的停滞。在发生缓存缺失的背景下，能够预测处理器下一个访存操作所需信息，并将该信息预先取回的预取技术就具有极高的工程研究前景与现实应用价值。预取技术具有设计可行性高、硬件开销较低以及应用范围广等优势，经过多年的发展，其已经被证明是隐藏处理器与主存之间访存延时的一种行为有效的手段。

为了提升系统中程序的运行效率，降低处理器访存延时，本文提出一种在处理器工作时检测并识别程序中数据流访存模式的硬件缓存预取策略。全文首先介绍了现有预取策略的特性并总结了各自的优缺点，紧接着对内存中程序访存行为特征进行了分析，如地址翻译方式、局部性原理、程序中基本访存模式等，同时还结合了程序实例探讨了预取技术所能带来的性能优化。然后给出了本文硬件预取方案中的预取地址结构、预取部件的位置、结构与组成，并详尽阐述了各个预取子模块所要实现的功能与工作原理。本文预取策略设置了多个数据流预取条目，以此达到识别并记录多个数据流的能力。同时，该策略还对程序中的顺序数据流与跨步数据流进行分开独立检测，最大限度避免了不同步长数据流之间的干扰，也提升了预取部件的准确性与效率。此外，为了防止预取请求影响正常指令的执行操作，本文还设置了预取请求缓冲暂存生成的预取请求，并制定了预取请求仲裁上访存流水线的优先级方案。

最后，在国产自主指令集架构处理器中完成本文预取部件的实例化，形成性能评估所需的完整处理器模型。使用 SPEC CPU2006 作为测试程序，并在硬件仿真加速器上进行实际仿真测试。结果表明，本文基于程序中数据流访存模式的预取策略在程序运行期间能够有效提升处理器的执行效率，增加各级缓存的命中率，最终实现降低处理器与主存之间访存延时的目的。

关键词：预取技术，硬件预取，局部性原理，访存模式，数据流

Abstract

Since the 1970s, the development trend of processors has always focused on how to improve the execution efficiency of instructions in the core, while the main memory has mainly focused on the increase in storage capacity, ignoring the increase in speed. The difference in the development trend of processors and main memory makes it difficult to match the memory access speed between the two, which directly leads to the "memory wall" problem that affects computer performance. In an attempt to bridge the speed difference between the processor and main memory, computer architects generally adopt a hierarchical storage structure that inserts multi-level caches in the system. However, the capacity of the cache is limited compared to the main memory after all. Once a cache miss occurs during the memory access, it will cause a large miss penalty delay, which will cause the processor to access the memory stagnation. In the context of cache misses, the prefetching technology that can predict the information required for the next memory access operation of the processor and retrieve the information in advance has extremely high engineering research prospects and practical application value. Prefetch technology has the advantages of high design feasibility, low hardware overhead, and wide application range. After years of development, it has been proven to be an effective means of hiding the memory access delay between the processor and the main memory.

So that improve the productivity of the procedure in the system and decrease the latency of the processor's memory access, this paper proposes a hardware cache prefetch strategy that detects and recognizes the data flow access mode in the program when the processor is working. The full text first introduces the feature of the existing prefetch strategies and condense their respective benefit and drawback, and next evaluate the feature of the memory access behavior of the program in memory, such as the address translation method, the principle of locality, the basic memory access mode in the program, etc. At the same time, it also discusses the performance optimization brought by prefetching technology in combination with a program example. Then it gives the prefetch address structure, the location, structure and composition of the prefetch components in the hardware prefetch scheme of this article, and elaborates the functions and working principles of each prefetch submodule in detail. The prefetch strategy in this article sets multiple data stream prefetch entries to achieve the ability to identify and record

multiple data streams. At the same time, the strategy also performs separate and independent detection of sequential data streams and stride data streams in the program, avoiding interference between asynchronous long data streams to the greatest extent, and also improving the accuracy and efficiency of prefetching components. In addition, in order to prevent prefetch requests from affecting the execution of normal instructions, this article also sets up prefetch requests to buffer and temporarily store prefetch requests, and formulates a priority scheme for prefetch request arbitration on the fetch pipeline.

Finally, complete the instantiation of the prefetch components in this article in a domestic autonomous instruction set architecture processor to form a complete processor model required for performance evaluation. Using SPEC CPU2006 as the test program, the actual run is performed on the hardware emulation accelerator. The results show that the prefetching strategy based on the data stream memory access mode may adequately enhance the execution speed of processor, raise the hit rate of all levels of cache, and finally diminish the memory access delay between the processor and main memory.

Key words: Prefetching, Hardware prefetch, Principle of locality, Memory access pattern, Data Stream

目 录

摘 要.....	I
Abstract	II
第一章 绪论	1
1.1 课题背景与研究意义	1
1.2 研究现状	3
1.2.1 IBM Power 系列处理器	4
1.2.2 Intel Core 系列处理器	5
1.3 本文主要工作	5
1.4 本文组织架构	6
第二章 预取技术概述	8
2.1 软件预取技术	9
2.2 硬件预取技术	11
2.2.1 顺序预取	11
2.2.2 流缓冲预取(Stream Buffer)	13
2.2.3 偏移量预取	14
2.2.4 马尔可夫预取(Markov)	15
2.2.5 帮助线程预取	17
2.3 软硬件混合预取技术	17
2.4 预取技术对比	18
2.5 本章小结	19
第三章 内存中程序访存行为特征分析	20
3.1 处理器访问内存方式	20
3.2 程序局部性原理	22
3.3 内存中应用程序的访存模式	23
3.3.1 基本访存模式	23
3.3.2 数据流访存模式实例	25
3.4 本章小结	27

第四章 基于数据流访存模式的硬件预取实现.....	28
4.1 预取地址.....	28
4.2 预取部件的位置.....	29
4.3 预取部件的结构与组成.....	30
4.3.1 预取控制模块.....	31
4.3.2 数据流预取条目模块.....	32
4.3.3 跨步数据流检测模块.....	34
4.3.4 预取请求缓冲.....	35
4.4 数据流访存模式的识别示例.....	36
4.4.1 顺序数据流的识别.....	36
4.4.2 跨步数据流的识别.....	38
4.5 本章小结.....	39
第五章 性能评测与数据分析.....	40
5.1 测试环境.....	40
5.1.1 性能测试基准程序(Benchmark).....	40
5.1.2 处理器模型.....	42
5.1.3 硬件仿真加速器.....	43
5.2 测试与评估方案.....	45
5.3 测试结果与分析.....	46
5.4 本章小结.....	51
总结与展望.....	53
参考文献.....	55
致谢.....	59

第一章 绪论

1.1 课题背景与研究意义

随着计算机微体系结构的创新与集成电路制造工艺的迅猛发展，芯片中指令访存的速度和核心逻辑单元的执行效率得以稳步上升，这最终促进了系统整体性能与工作效率大幅度提升。国内外许多研究表明，自 1980s 以来，CPU(中央处理器)执行速度一直呈指数型增长，每隔一年性能上升超过 50%。而主存储器受限于自身电容密度等因素，其内部存储单元每一年的访存速度仅增加不到 10%^[1]。如果再考虑到新技术的运用，如处理器采用更加激进的超标量、超流水线以及超长指令字等设计，则处理器与内存之间的性能差距可能更大。处理器与主存之间的巨大速度差异直接导致了令研究人员棘手的“存储墙(Memory Wall)”问题^[2, 3, 4]。下图 1.1 清楚的显示了近几十年处理器与内存之间速度提升的不平衡性。

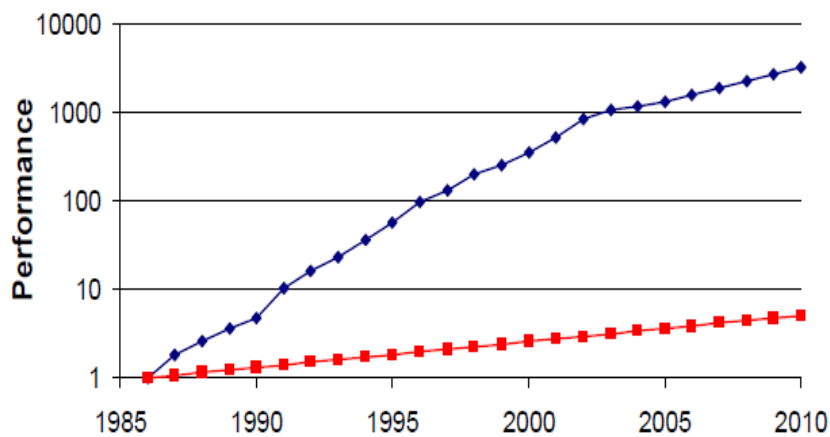


图 1.1 处理器与主存之间性能增长的对比(蓝色：处理器；红色：主存)

现代计算机系统结构中，经典的冯诺依曼型机主要由数据供给系统与数据处理系统两部分组成^[5]。计算机系统运行程序时，处理器需要不断从存储系统获取指令与数据并执行相应的操作，最后再将处理之后的数据写入存储系统中。在理想条件下，当数据供给与数据处理达到动态平衡时，计算机的性能就得以实现效率最大化。然而，在实际情况中，由于“存储墙”问题的出现，这种数据供给与数据处理的动态平衡难以建立。当程序访存过程中处理器与主存之间进行信息交换时，系统需要耗费大量额外的时钟周期用来等待数据从主存中送入处理器，结果导致整个系统的访存时间几乎完全取决于内存

向处理器发送数据的速度。因此，如何减轻甚至消除“存储墙”效应，降低系统访存延时就成了当前研究的热点问题^[6]。

为了最大程度缓解“存储墙”问题所带来的处理器与内存之间巨大的访存延时，计算机架构师们普遍采用如图 1.2 所示“金字塔”型的层次存储结构来试图弥合处理器与主存之间的速度差^[7]。该结构中，从最靠近 CPU(Central Processing Unit)的顶层往底层走，存储设备变得速度更慢、容量更大和成本更便宜。在金字塔顶，是处理器内核的各类寄存器文件，程序运行时 CPU 仅需单位时钟就能获取其中的数据。接下来是一个由 SRAM(Static Random Access Memory)构成的三级高速缓存存储器(Cache)，当发生缓存命中时，CPU 可以在几个到几十个时钟周期内访问 Cache，这在很大程度上缓解了“存储墙”所带来的访存延时。然后是一个大的基于 DRAM(Dynamic Random Access Memory)的主存储器，主存与 CPU 之间的指令或数据传输速度往往能够消耗上百个时钟，与 Cache 相比，主存的访存延时十分巨大，这对于现代高性能处理器来说是无法接受的。再往下走是慢速但是容量巨大的外部存储器，其主要由磁盘构成。最后，离 CPU 最远的是独立于计算机之外的远程二级存储，它们的访问要利用内部局域网或者互联网来进行，一般存在于大型分离式系统中。

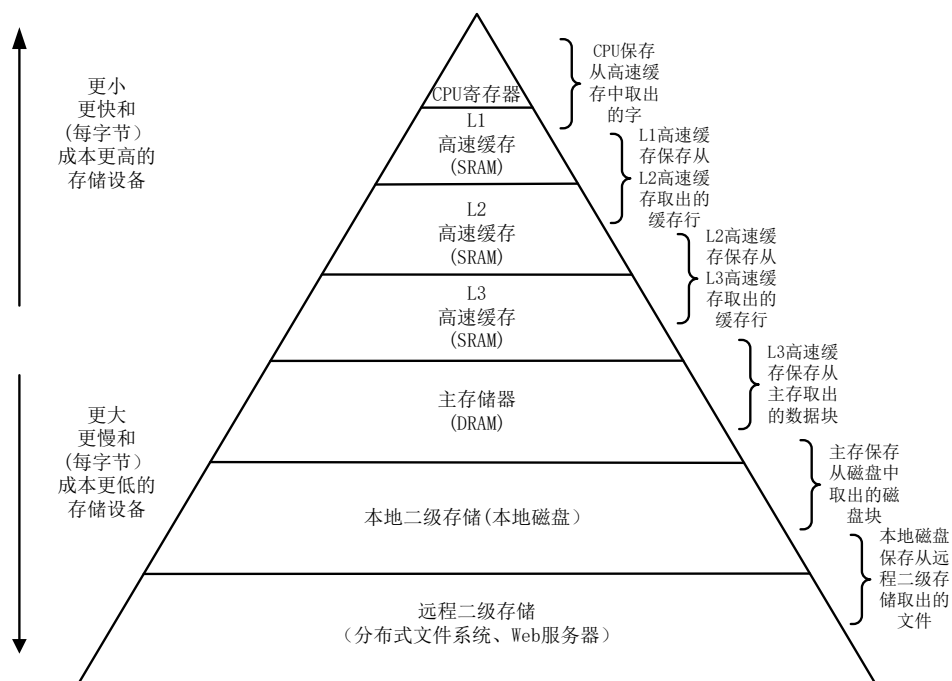


图 1.2 存储器层次结构

层次存储结构降低系统访存延时的关键在于多级 Cache 的插入。依据程序访问的局部性原理，主存中被处理器访问过的数据或与被访问数据相邻的数据元素往往以副本的

形式保存在 Cache 中，这样当处理器执行下一个操作时，就有很大概率能快速读取所需的数据或指令。在数据访存速度方面，L1Cache(Level 1 Cache)的访问速度几乎与处理器内部的寄存器一样快，当访存命中 L1Cache 时，其访存时间大约为 4 个时钟周期，处理器内部流水线等待时间很小。当 L1Cache 访存失效时，处理器也能够 在 10 个时钟周期内访问到下一级的 L2Cache(Level 2 Cache)。现代存储系统往往还含有一个更大的高速缓存，称为 L3Cache(Level 3 Cache)或 LLC(Last level Cache)，在前两级 Cache 发生失效时，处理器访问 L3Cache 也仅需要大约 50 个时钟周期。然而，缓存的容量毕竟有限，处理器访存时无法保证所需的指令或数据都能直接在 Cache 获取。一旦出现 Cache 访存缺失，再去从主存取回数据将会消耗数百个单位的时钟，这一过程就不可避免的造成处理器长时间的停滞。

业界用来评估处理器与存储系统之间访存效率的指标参数可表示为：平均访问时间 = 命中时间 + 失效率 * 失效开销。从公式看，处理器完成一次访存操作除了最基本的命中延时外，如果发生访存缺失，还会多出由失效率与失效开销共同决定的额外延时。因此在层次型存储结构下，除了要尽量优化系统中的访存路径与结构来降低命中时间，例如，设计结构简单的 L1Cache、采用多级页表与多级 TLB(Translation Lookaside Buffer)、增加指令轨迹 Cache、虚拟 Cache 等技术。还应着重解决访存过程中 Cache 失效所带来的性能损失，例如，利用增加数据宽度、提前启动、读缺失优先仲裁、优化请求字处理方式、等技术用来缓解 Cache 失效时的开销；而通过有限度提高缓存容量、增加 Cache 组相连度、预取、设置一个存放被替换数据块的 Cache(Victim Cache)、列相连、编译器优化等技术则用来降低 Cache 失效率。

在现有所有缓解“存储墙”问题的技术当中，致力于提高 Cache 命中率的预取技术一经提出就得到了广泛的研究与讨论。预取技术具有应用范围广、运用灵活、设计复杂度低的优势，同时也能够根据各种应用程序的特征做出相应的优化。从多重层面综合分析来看，预取技术是目前隐藏系统访存延时最有效，且最具研究前景的一种方法。

1.2 研究现状

预取技术经过几十年的发展与深入研究，许许多多针对不同领域的预取机制相继被提出。然而，基于硬件设计实现难度与系统加速效果之间的考虑，行业标杆主流商业处

理器供应商在产品中，除了提供基本软件预取指令外仍然采用顺序预取、流预取等结构相对易实现的硬件预取机制。

1.2.1 IBM Power 系列处理器

Power 系列 CPU 是 IBM 机构基于精简指令集所设计的芯片。其主要面向于超算与服务器领域。每一代 Power 处理器的结构中都能够看到预取部件的身影^[8,9,10]。

2001 年问世的 Power4 处理器中，除了插入允许程序指定预取方向、预取深度的预取提示指令 `dcbt` 或 `dcbtst` 外，还在 L1Cache 中搭配了对软件可见的硬件预取部件。Power4 访存过程中，当发生 L1Cache 缺失时，就从下一级 Cache 中预取识别出的数据。整体来看，Power4 预取设计简单且效率不高，预取数据的识别主要依靠 8 个流预取部件。下一代 Power5 中硬件预取部件与 Power4 大致相同，每个处理器中仍保留 8 个流预取部件，区别之处在于 Power5 对预取提示指令 `dcbt` 进行了拓展，分为了读预取提示指令与写预取提示指令两种。

Power6 在之前的基础上将数据流的识别条目增加到了 16 个，可以实现同时对 16 个数据流的检测。为了最大程度提升预取数据的及时性，L1Cache 与 L2Cache 预取部件一次取回的 Cache 块的数量由新增的流控制器来动态调整。Power7 大致沿用 Power6 的整体设计，处理器中设有可编程的硬件数据预取器。Power8 中采取了更加积极的 (Aggressive) 预取机制，预取部件的设计也更加复杂。预取部件放置在 Load/Store 存储单元中以利用更为丰富的访存信息，工作原理依旧为对流访问特征的识别。此外，Power8 中还增加了数据预取深度感知(Data prefetch depth awareness)、指令推测感知(Instruction speculation awareness)、自适应带宽与拓扑感知(Adaptive bandwidth & Topology awareness) 等技术。

2018 年 IBM 发布的 Power9 处理器对硬件预取过程中的及时性问题与减轻 Cache 污染做出了进一步的优化。在自适应预取(Adaptive Prefetching)方面，Power9 根据程序访存过程中的历史信息与识别的数据流，确定每个预取请求的置信度(Confidence level)，在内存带宽较低时，内存控制器优先选择置信度较高的预取请求。与此同时，Power9 中预取部件还会接收来自内存控制器的反馈，从而动态的确定预取的深度。

2020 年 8 月，IBM 公司发布了其最新一代，采用三星半导体先进 7nm 制程工艺的 Power10 处理器。据资料显示 Power10 处理器在综合能效、工作负载以及容器密度等方

面比 Power9 提升了近 3 倍。Power10 核心中设计有智能预取机制,其详细工作原理、地址预测策略以及预取部件的开销,有待 2021 年四季度该芯片进入市场后进一步公布。

1.2.2 Intel Core 系列处理器

与 Power 系列处理器不同,Intel Core 系列处理器基于各级 Cache 容量、访存延时等特性的差异,分别对 L1Cache 与 L2、L3Cache 设计了不同的硬件预取部件,其中每个处理器中都包含了两个 L1Cache 预取器与两个 L2、L3Cache 预取器^[11,12]。

检测程序访问方式的硬件逻辑 DCU(Data Cache Unit)与依据访存地址相关性而制定的指针逻辑,是设计在离处理器较近的一级 Cache 中的两种预取部件。DCU 与其它流预取部件类似,通过对程序访存过程中数据流的识别,并且动态确定所要预取 Cache 行的数量。指针预取器对访存指令的信息进行追踪并且保存在一个地址阵列中,随后通过对指令历史信息分析,该预取器预测出下一指令的访存地址,并发出预取请求,最大检测步长为 2K。

对于层次相对较低的 L2/L3Cache,Intel Core 处理器设计有 Spatial Prefetcher 与 Streamer 两种预取部件。Spatial 预取器通过空间地址的相关性(多个内存页面之间访问模式的相似性)来预测将来的内存访问。例如,如果程序访问了页面 A 的位置{X, Y, Z},则它很可能在将来访问其他相似页面的{X, Y, Z}位置。Streamer 则通过对来自 L1Cache 的访存失效请求进行检测,当检测到正向或逆向的数据流时,触发对后续的 Cache 行进行预取。其中,预取的 Cache 行数据必须在大小为 4K 的物理页当中,不允许跨页对数据进行预取。

最后,Intel Core 处理器中还设计有一个预取监控装置 PM(Prefetch Monitor),其主要功能为监控处理器访存过程中的数据带宽流量。当系统中总线压力过大时,PM 就会控制预取请求的数量,从而很好的平衡了总线带宽压力与预取部件效率。

1.3 本文主要工作

本文研究目的旨在以程序中大量存在的数据流访存模式为前提,在国产自主指令集架构处理器中完成预取策略的实现,希望以此加速程序的执行效率,从而最大限度的隐藏处理器与主存之间的访存延时。基于上述目标,全文完成工作如下:

- 1、首先对预取技术的相关知识点进行介绍,例如,不同预取策略的实现方式与优缺

点、处理器访问内存数据的寻址方式、程序局部性原理以及程序中存在的访存模式。最后，结合程序代码讨论了数据流访问中预取技术的可行性。

2、给出了国产处理器的 64 位虚拟访存地址结构，并对地址中不同字段在访存中所要完成的功能进行介绍，给出了本文基于 Cache 行地址来对程序中数据流访存模式进行检测的原则。同时还对国产处理器的访存路径进行了分析，讨论了一条指令从发出到实际访问数据所要执行的一系列操作，最终确定在处理器访存流水线中的数据 Cache 管理单元中添加预取部件。

3、在国产处理器内核的数据 Cache 管理部件中，使用硬件描述语言完成预取部件的设计并例化。预取部件中分别设计检测顺序数据流与跨步数据流的模块，并利用预取控制模块统一指导预取。同时为了避免预取请求影响正常指令的执行，设置预取缓冲条目暂存生成的预取请求，并制定预取请求上流水线的仲裁策略。

4、搭建评估处理器性能的验证环境，使用 SPEC CPU2006 作为测试程序，以硬件仿真加速器为验证工具。从结果来看，本文预取部件能够有效的识别出程序中数据访存模式，提升处理器访存时指令的执行效率与缓存命中率。

1.4 本文组织架构

本文内容分为五部分，各章内容如下：

第一章 绪论

本章着重介绍了预取技术产生的背景，阐述了本课题对解决“存储墙”问题的实际意义，接着分别以 Intel 与 IBM 公司商用处理器中预取技术的发展轨迹显示了本课题的研究现状，最后，简明总结了全文完成的工作与文章架构。

第二章 预取技术概述

本章主要是对预取技术做一个较为全面的叙述，分别就现有预取策略的工作原理、设计方式与适用的条件进行了分析与讨论，最后，总结出三类预取策略的优缺点。

第三章 内存中程序访存行为特征分析

本章首先叙述了内存系统中数据的寻址方式，并分析了处理器实际访问内存数据时的地址翻译过程。接着讨论了程序的局部性原理。最后详细介绍了程序运行时所表现出的几种访存模式，并结合程序代码示例表明预取能够有效提升程序中数据访存模式的执

行效率。

第四章 基于数据流访存模式的硬件预取部件的实现

本章在前面章节的分析的基础上，设计了基于数据流访存模式的预取部件，旨在对顺序数据流以及跨步数据流做出准确识别。同时对预取地址、预取部件的位置、预取部件的结构与组成等内容进行了详尽的介绍。

第五章 性能测评

本章首先确定了性能评估所需的必要环境，主要包括处理器模型、测试程序与仿真加速平台。制定了性能评估方案，选择了 IPC 与各级缓存命中率这两个重要参数来反映预取部件的设置对处理器性能产生的影响。最后，对仿真结果进行了分析并得出最终结论。

第二章 预取技术概述

预取技术是一种预先获取处理器所需数据的技术。预取部件在指令发生访存失效时对将来可能访问的数据进行预测并发出预取请求,以便在该数据真正被使用之前送入缓存或特定预取数据缓冲,从而避免因访存失效造成处理器停顿,达到提高系统运行性能的目的^[13]。通过硬件平台与软件指令的支持,预取技术已被广泛认为是隐藏内存访存延时的一种行之有效的方法。下图 2.1 给出了预取技术降低处理器访存延时的一个示例。

图 2.1 中三种访存示例完成同一个任务: 其中示例 a 为无预取的普通访存操作; 示例 b 表示理论条件下的预取; 示例 c 则代表了实际执行中预取的效果。处理器完成任务假定分为 4 个阶段, 各自阶段中又包括逻辑运算与访问内存两部分, 而图中上方数字刻度则显示了完成各个访存操作所占用的时钟周期数。

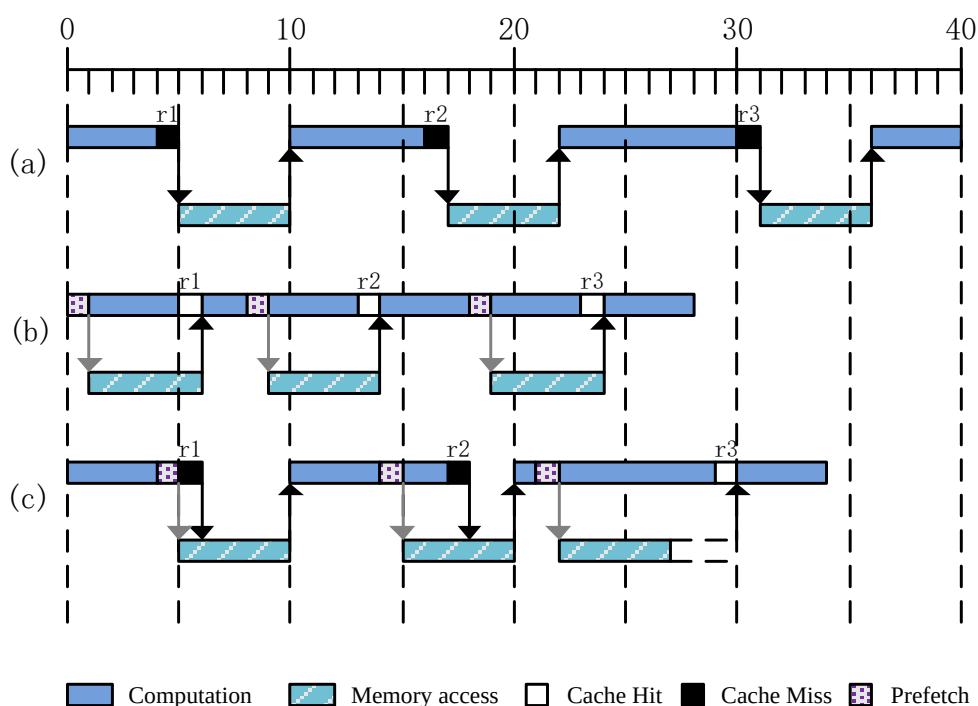


图 2.1 预取技术实例: a)没有预取 b)理想情况预取 c)实际情况下预取

示例 a 中无预取指令或预取部件, 在指令执行时, 无法保证处理器所需的数据都能在 Cache 中获取, 即可能发生 Cache Miss 的情况。如在 r1、r2、r3 发生访存缺失, 此时处理器就需要对存储器进行访问, 带来额外的访存延时, 最终花费 40 个时钟周期才能完成示例任务。示例 b 为增加预取模块, 且预取时机完美的模型。程序执行时, 预取模块会使用一个时钟周期将数据送入缓存, 尽管预取操作会单独占用一个时钟周期, 但却实现了处理器与主存储器之间最大的并行。对预取操作的合理使用, 处理器理论上只要

花费 28 个时钟周期就能实现相同的功能，从根本上避免了任何形式的访存延时。示例 b 的访存结果是在完全理想的情况下得出的，系统在运行程序中一直是首尾连贯无间断的，CPU 对任务的执行与内存的访问也处于高度并行的机制下。然而以实际角度来看，示例 b 存在的概论较低。预取时机的确定、预取数据的使用以及从何处预取等因素都将对预取的实际效果产生影响。接着看示例 c，第一阶段预取时机太晚，因此处理器执行任务时仍然会出现 Cache Miss 的情况；第二阶段中，预取的时机稍微提前，虽然访存操作依旧会出现 Cache Miss，但访存延时却有所降低；在第三阶段中，预取操作产生的过早，预取数据提前进入了 Cache 中并替换了 Cache 块中的数据，后续访存操作命中 Cache，但却有可能发生 Cache 污染，因为被替换出去的 Cache 块很有可能在接下来会被处理器所访问。实例 c 共用时 34 个时钟周期，与示例 c 相比，也能较好的提升系统整体性能。

预取技术按照其实现方式主要可以分为以下三种：软件、硬件、以及软硬件混合预取^[14]。本文主要关注数据预取技术的实现机制，下面将从工作原理、实现方式等方面对这三种预取技术进行详细介绍与分析。

2.1 软件预取技术

早在上世纪开始，软件预取就以某种形式存在了，第一个提出缓存获取指令(Cache fetch instruction)概念的人是 Porterfield，但当时对于软件预取的有效性还是持怀疑态度^[15]。经过几十年的发展，当前国内外大多数现代微处理器都支持某种形式的软件 fetch 指令，这些指令则可以用来对数据进行预取。软件方法的主要策略手段是在编译时由系统或者芯片工程师向程序中增加功能代码，以实现在下一个 Load/Store 指令到来之前将数据存入 Cache 中。

最基本的软件预取实现方式是在应用程序中插入一段预先获取数据的指令代码段，当程序执行时，在处理器需要数据之前，就通过预取指令提前将目标数据从缓存或主存中送入内部寄存器。例如，在链表数据结构中，编译器可以在访问结点的子结点插入预取指令来触发预取请求^[16]。稍微复杂一点的软件预取实现当中，预取指令代码可能还会提供对预取数据块如何使用的提示，这样的信息一般用在多核共享的数据当中。

许多研究表明，无论是编程人员手动加入指令还是借助编译器自动进行优化实现，程序中仅加入一些预取指令便可以大幅度的提升处理器的性能^[17]。软件预取一般应用于

大型数组与大规模向量中的 Loop 循环，通过在编译时将预取指令内嵌入循环体中，从而达到程序在当前迭代期间预取将来用于循环的数据。下图 2.2 给出了软件预取指令的一个实例，其中，图 a 为原始代码，图 b 为在循环体中插入软件预取指令后的代码段。

<pre>int x[N]; int y[N]; int z[N]; for(int j=0; j<N; j++) { z[j] = 3*x[j] + y[j]; }</pre>	<pre>int x[N]; int y[N]; int z[N]; for(int j=0; j<N; j++) { PREFETCH(&x[j*k], k); PREFETCH(&y[j*k], k); for(int i=0; i<k; i++) { z[j*k+i] = 3*x[j*k+i] + y[j*k+i]; } }</pre>
a) original code	b) code with prefetch instructions

图 2.2 软件预取代码实例

一般情况下，程序中软件预取指令的加入只会对计算机系统的整体性能产生影响，而对处理器工作的正确性不造成干扰，更不会导致应用程序访存过程中出现诸如内存错误等其他内存异常情况^[18]。软件预取技术中插入了大量的预取指令代码段，这些指令在分析程序结构之外，还要在同一时间完成后续访存地址的计算。因此软件人员与编译器都无法保证在 Load/Store 执行时及时预取回所需处理的数据，往往难以做到完全隐藏访存延时。软件预取另一个值得改进的点是，在过多预取指令带来程序代码冗余度增加的同时，也要尽量降低处理器内核的电路开销(避免占用过多的内部寄存器文件)，力求整个系统的成本达到最低^[19]。虽然软件预取达不到理性情况下的预取时效性，但当前大多数主流的商用处理器中都插入了专门的预取指令，下表 2.1 对此做出了统计。

表 2.1 部分主流商用处理器预取指令

处理器类型	预取指令
Motorola88110 处理器	Touch Load
Power PC 系列处理器	DCBT
Intel 系列处理器	Dummy Read 与 PREFETCH
ARM 处理器	PLD 与 PLI
Alpha 体系结构处理器	FETCH 与 FETCH-M
MIPS 处理器	R10000

2.2 硬件预取技术

通过在内核中设计具体的逻辑电路(如缓冲条目、记录表等),来对处理器一段时间内产生的读/写操作所命中的地址进行检测,并根据实际检测结果预测下一次访问地址,动态的将低层次存储器中数据预先取入更高层次存储器的技术称为硬件预取^[20]。硬件预取技术的实现主要依赖于程序的局部性原理或系统中丰富的并行硬件资源。与软件预取相比,硬件预取不需要编程人员或者编译器产生额外的代码,软件在预取的过程中不发挥作用,更不用浪费一条预取指令来将数据装入 Cache 或特定缓冲当中。硬件预取机制是降低处理器的访存延时的另一有效设计途径,大多数主流的商用处理器中都设计有某种形式的硬件预取部件。

2.2.1 顺序预取

顺序预取是设计中最为简单的一种硬件预取机制,其可行性主要依赖于程序的空间局部性。早在上个世纪的 60 年代,IBM 公司的 System/360 Model 91 处理器就第一次实现了这种预取策略。顺序预取又称为 Next-Line 预取或 OBL(One Block Lookahead)预取机制^[21]。OBL 按照工作原理又可分为三种实现方式:总是预取(Always Prefetch)、访存缺失预取(Prefetch On Miss)以及标记位预取(Tagged Prefetch)^[22]。

Always Prefetch 预取的工作原理是:当处理器访问程序的数据块 i 时,只要数据块 $i+1$ 不在当前的 Cache 中,就将数据块 $i+1$ 从低层次的 Cache 中预取过来。由于 Always Prefetch 机制没有为预取来的数据设置额外的缓冲条目,如果预取过来的数据块 $i+1$ 在随后的一段时间内没有被访问,就会造成预取数据浪费的情况,同时该数据还会挤占 Cache 中使用概率较高的数据的位置。Prefetch On Miss 预取机制的实现方式为:当数据块 i 访问失效时,首先将数据块 i 取入 Cache 供处理器操作,随后再根据数据块 $i+1$ 是否在 Cache 中决定是否发出预取请求。Prefetch On Miss 预取与 Always Prefetch 相比降低了 Cache 的概率,但这两种方法存在对访存历史信息使用不充分的情况。

Tagged Prefetch 是在 Prefetch On Miss 的基础上通过增加额外硬件资源的一种改进。Tagged Prefetch 在每个 Cache 块前增加一个标记位(Tag)来实现访存信息对发出预取请求的指导。如下图 2.3 所示,Cache 块中标记位在处理器复位以及 Cache 块中数据被替换时设置为 0,当块中预取获得的数据第一次使用时标记位由 0 变为 1,此时如果下一相

邻地址的数据不在 Cache 中，则触发预取将此数据取回。Tagged Prefetch 与 Prefetch On Miss 的主要区别在于访问预取获得数据时是否触发预取：Prefetch On Miss 中无论后续地址的数据是否在 Cache 中都不会触发预取，而 Tagged Prefetch 机制在访问预取而来的数据时，首先标志位置 1，紧接着会对地址相邻的下一数据进行判断，如果该数据不在 Cache 中则发出预取请求。

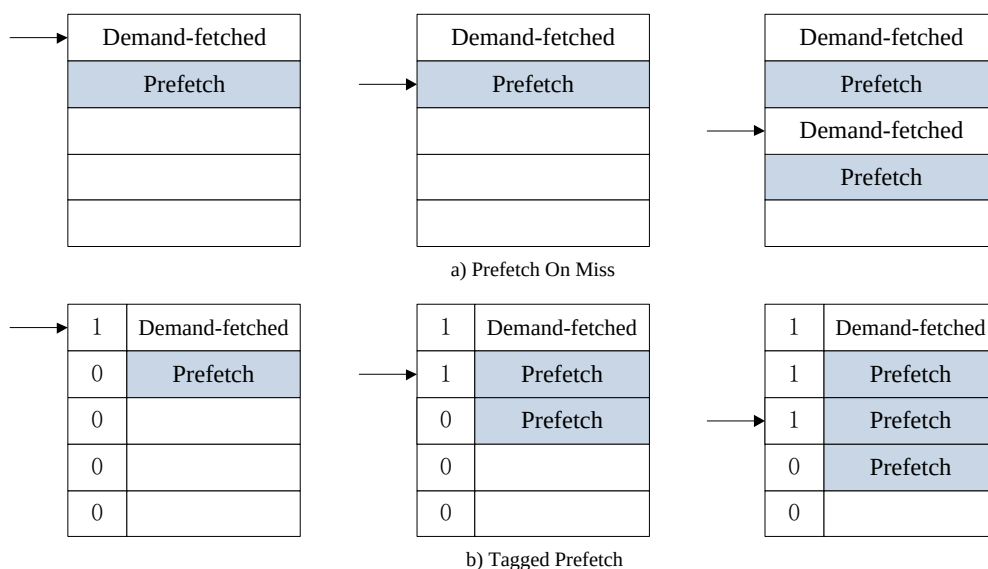


图 2.3 Prefetch On Miss 与 Tagged Prefetch 两种预取机制比较

OBL 方式预取的数据块为一个，对于访存间隔周期较短的指令来说，经常会出现预取数据无法及时送入处理器执行的情况。IBL 预取策略(Infinite-Block Look-ahead)是对 OBL 的改进，当数据块 b 被访问时，预取部件实现对数据块 b 之后的 $b+k$ 块进行预取，其中 k 为预取深度^[23]。相比于 OBL 预取技术，该方法增大了数据块的预取深度，提高了预取的效率。但 IBL 有一个问题值得关注，当 k 的值过高后，过多的数据会导致总线带宽压力增加与 Cache 污染。针对过度预取的问题，Dahlgren 等人则提出一种自适应预取数据的顺序预取策略^[24]。该策略允许预取深度 k 在程序执行中动态的变化，以使预取的深度与程序本身所展现的空间局部性相匹配。为此，缓存会定时计算处理器的预取效率，并作为程序空间局部性优劣的指标，预取效率则用访存请求命中预取 Cache 块与总预取的比值来表示。与其它形式的顺序预取相比，自适应预取能够显著降低缓存的未命中率，但是该策略的硬件实现较为复杂。

顺序预取不会对程序的整体结构做出分析，也没有充分利用访存中产生的丰富历史信息，其预取的效率与准确性完全取决于程序的空间局部性。在空间局部性较好的结构

中，其可以解决处理器停顿的问题；但对于空间局部性不是很理想的程序来说，顺序预取所表现出的盲目性，以及预取过程中所带来的无用数据与带宽浪费，同样不可忽视。

2.2.2 流缓冲预取(Stream Buffer)

Stream Buffer 预取是一种利用额外存储结构来保存预取数据的硬件预取技术^[25]。与主存相比，Cache 容量十分有限，并且 Cache 中数据被访问的几率更高，因此 Cache 中的有用数据一旦被替换，结果带来的损失是难以接受的。Stream Buffer 结构能够很好避免上述问题的出现，其硬件设计通过 FIFO(First In First Out)存储方式实现。在 Stream Buffer 预取机制中，处理器将预取而来的数据首先存入 Stream Buffer 中而不是直接放入 Cache，这样就将预取来的数据与 Cache 中原本的数据进行了分离，很好的解决了由于 Cache 中数据块频繁的替换所带来的 Cache 污染。

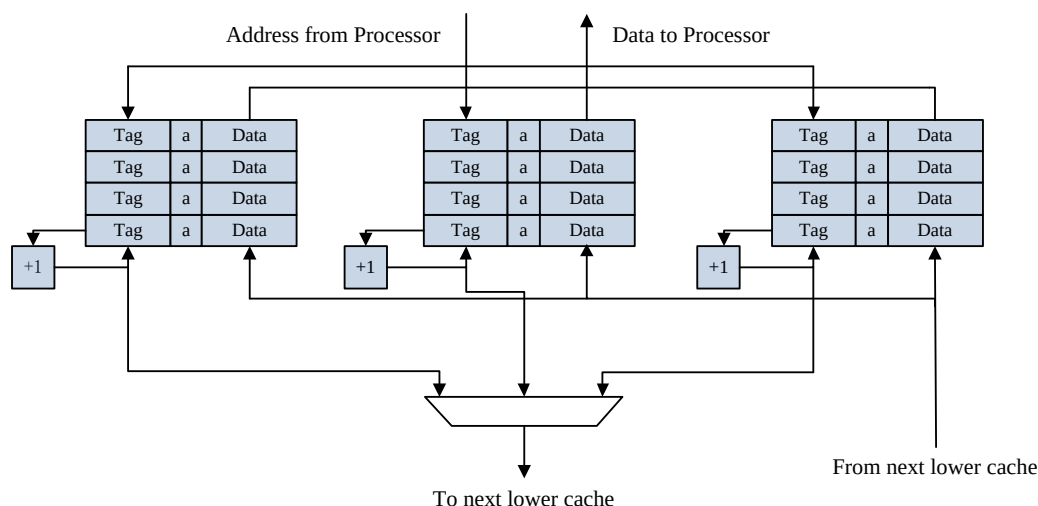


图 2.4 Multi-way Stream Buffer 预取机制结构图

从图 2.4 中可以看出，每个 Stream Buffer 由多个条目组成，每个条目中又包括了 Tag 标记位、有效位 a，以及 Data 数据块。当处理器发出访存操作时，通过地址的 Tag 位同时对 Cache 与 Stream Buffer 进行查找，如果访存地址直接命中 Cache，那么 Cache 中的数据将会送入处理器执行单元进行操作。当发生 Cache 失效且命中 Stream Buffer 时，这时命中的 Stream Buffer 中第一个条目送入 Cache 中，同时后续条目向缓存的头部移动，低层次存储器中预取的数据也将填充至 Stream Buffer 尾部空闲的缓冲条目。如果出现 Cache 与 Stream Buffer 同时不命中的情况，那么按照 FIFO 先入先出的存储特性，则利用 LRU(Least Recently Used)替换算法选择其中一个 Stream Buffer 进行清空，并根据预取部件识别出的 Stream 访问特征发出预取请求。

Stream Buffer 预取中设置有独立的缓冲区,以 Cache 与独立缓冲之间较低的数据传输延时为代价,来避免传统预取中可能出现的 Cache 污染问题。但由于数据单独保存在独立的存储缓冲中,随之而来会出现缓存数据一致性的问题,同时也增加了硬件电路设计的复杂性。其次,当处理器所需数据在 Cache 与 Stream Buffer 中均不命中时,系统会先对整个 Stream Buffer 条进行清空并重新预取新的数据块,这也会造成预取数据的与总线带宽的浪费。最后,传统的 Stream Buffer 预取只能对单个数据流进行识别,预取部件整体的效率偏低。

基于上述所提出的缺陷与不足,Palachrala 与 Kessler 对存储系统组织进行了全面的评估,通过增加额外的硬件资源对预取地址进行过滤,并引入了去噪方案,提高预取的准确性与覆盖率^[26]。Kim 等人对程序执行过程中,访存时序信息的来源进行了系统性的分析,通过将访存地址划分为不同的 Stream 来减少不必要数据的预取,节约了宝贵的总线带宽^[27]。Zhang 和 MCKee 也对 Stream Buffer 预取的缺陷进行了分析并做出了相应优化^[28]。

2.2.3 偏移量预取

为了进一步降低处理器访存延时,近年来出现了基于访存过程中地址偏移量(Offset)的硬件预取机制^[29]。偏移预取思想来源于传统的顺序预取,当处理器访问地址为 X 的 Cache 块时,如果数据块 $X+D$ 不在当前 Cache 中,偏移预取器就会对该数据发出预取请求,其中 D 又被称为预取偏移量。预取部件通过对访存过程中记录的地址历史信息分析,设计特定训练算法,动态的推测出下一访问数据的地址与当前地址之间最有可能的差值,并以此作为预取偏移量。当偏移量 D 为 1 时,偏移量预取即为我们熟知的顺序预取中的一种形式。

在实际应用当中,由于应用程序空间局部性的优劣,不同程序潜在的预取偏移量必然是有差异的,因此一个成熟的偏移预取器应该能够通过对应用程序访存过程中的特征行为方式进行分析,确保在每次访存操作中动态的选择出最适合的预取偏移量。Sandbox 预取器是由 Pugsley 首次提出的一种偏移量预取策略,然而 Sandbox 预取器中的偏移量算法忽视了预取及时性的问题^[30]。为了解决上述问题,Pierre 等人在容量与访存延时适中的 L2Cache 中,设计了一种名为 The Best-Offset(BO)的偏移量预取机制,其基本结构如下图 2.5 所示^[31]。

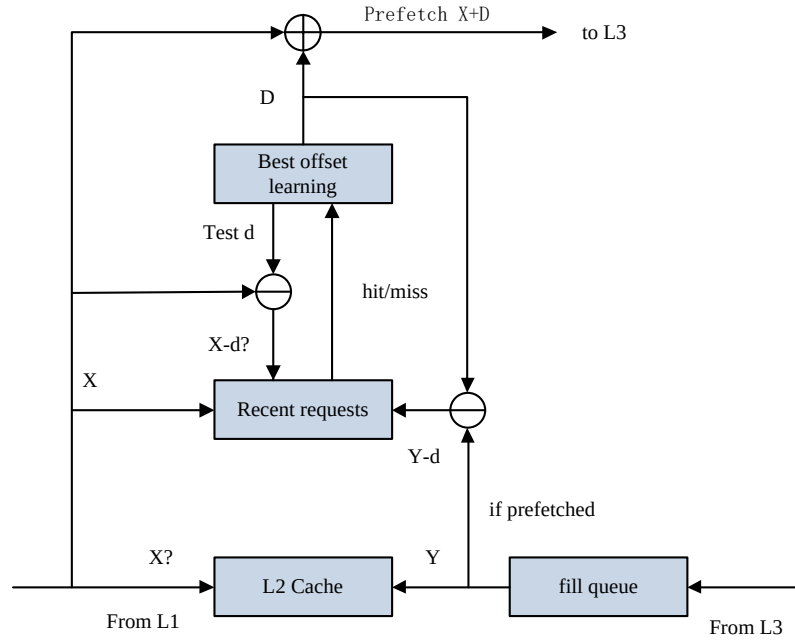


图 2.5 BO 预取结构图

BO 预取部件主要由最近请求访问表 RR(Recent Requests)与最佳偏移量选择模块(Best offset learning)组成。RR 表中记录访存失效与命中预取数据请求的地址，最佳偏移量选择模块由偏移量列表(Offset list)与偏移量分数列表(Score table)组成。当访存请求 X 失效或命中预取而来的数据块时，用地址 X 依次减去偏移量列表中的偏移量 $d_i(i=1,2,3,\dots,N)$ ，其中 N 为列表中偏移量 d 的个数；如果 $X-d_i$ 命中 RR 表中内容，则 Scores table 中对应的偏移量 d_i 分数增加。预取偏移量选择阶段结束后，Offset list 表中所有偏移量都完成了一遍检测，其中 Scores table 中分数最高的偏移量 D 即为最佳预取偏移量，预取地址则为 $X+D$ 。

偏移量预取在每次发出预取请求前动态的选择最佳偏移量，有效降低了硬件预取中存在的盲目性，其适用范围较广，能够在不同访存模式中发挥最大的效率。但是，为了在每次访存请求失效后选出最理想的预取偏移量，预取部件中需要维护一个极大的 RR 表与偏移量列表，这就带来了额外的硬件开销与设计复杂度。因此设计人员往往需要在预取效益与设计代价之间做出权衡，否则将会得不偿失。

2.2.4 马尔可夫预取(Markov)

上述几个小节主要介绍了几种依赖于应用程序空间局部性原理的硬件预取部件，接下来，本节将简要介绍一种基于时间局部性的硬件预取机制。基于时间局部性的预取主

要通过对一段时间内程序的访存特征进行分析,找出并记录其中具有规律性重复出现的访存序列链(地址按规律重复出现),当后续程序执行的过程中该访存地址链再次被访问时,依据此前记录的访存地址链信息对随后潜在访问的数据序列进行预取^[32]。

Markov 模型是最简单的一种依赖应用程序时间局部性原理的预取策略。Markov 预取器构造了一个地址访问序列表,表中的每个条目用来记录访存过程中出现的一条或多条重复出现的访存地址序列,一旦表中记录的访存序列发生 Cache 失效就触发预取并取回将来潜在访问的数据^[33]。从下图 2.5 中的示例可以看出,Markov 模型中记录了数条具有规律性的访存地址链,同时状态图还给出了序列中访问下一个潜在对象的概率。以第一条访存地址链为例:当处理器访问地址 7FFF8000 时,下一个潜在的访存对象的地址则为 1010FF00、7FFF8000 以及 10B0C600,并且访问这些数据的概率是相同的都为 1/3。当程序后续的执行过程中此访存地址链一旦被再次访问,预取部件将会依据该序列链获取数据。Markov 模型中记录的访存序列链越多,预取部件的覆盖率与发出预取请求的概率也就越大。

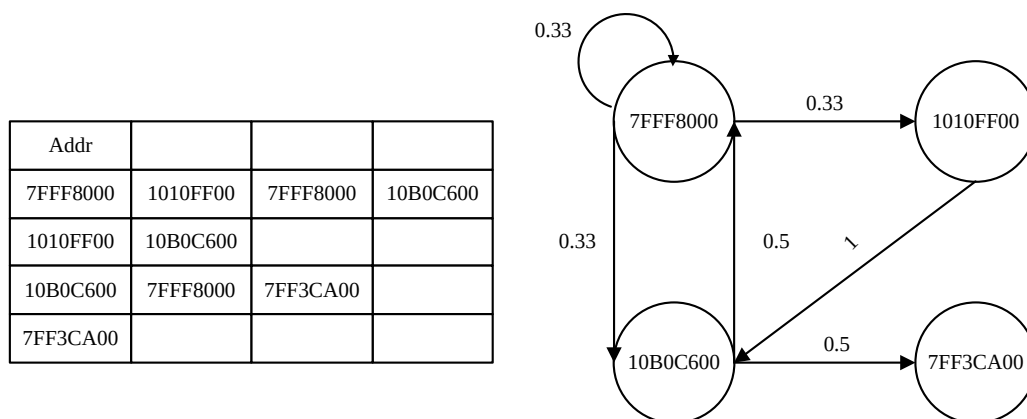


图 2.6 Markov 预取实例

Markov 预取对降低系统的访存延时有着一定的积极作用,但以下几点限制因素大大的制约了其可实施性:首先,应用程序访存模式中具有重复规律的情况有限,所以根据历史来预测的准确性较低;其次,具有相关性的访存地址链可能具有上百个并且每个访存序列的长度大小不一,Wensich 等人对商业处理器应用程序中具有时间局部性的访存地址链进行了统计,结果发现此种形式的访存序列长度分布跨度范围从 2 个 Cache 块到数千个 Cache 块不等,存储如此大历史信息的额外存储空间消耗是巨大的^[34]。最后,每个访存操作可能对应多个潜在的后续访存操作,如果对这些数据都进行预取的话可能会带来大量的无用数据导致 Cache 污染,也额外增加了数据总线的压力造成带宽的浪费。

2.2.5 帮助线程预取

充分利用额外的并行资源是硬件预取机制的另一种实现策略。现如今，多核与多线程技术已逐渐成为主流的处理器硬件平台^[35]。它们丰富的并行资源为提升硬件预取的效率与增强处理器性能等方面提供了技术的支持。利用额外并行资源的硬件预取器，其主要工作特点是运行速度快于正常访存指令，同时仍然能够通过地址计算来预取后续所需执行的数据。

在具有多线程的处理器中，当主线程运行程序时，另一个线程则可以执行该程序的精简版本，并提前获取后续所需的数据，这样的线程被称为帮助线程或预取线程^[36]。当处理器访存时，内核中两个线程并行处理任务，鉴于两个线程运行不同规模的同一程序，运行主程序子集的帮助线程自然会具备更高的效率。于是，帮助线程在访存过程中总是会提前得到处理器所要执行的数据，然后再通过存储系统中的共享 Cache 将提前获取的数据传递给主线程，最终实现数据预取的目的。然而，系统中共享 Cache 数据传输的速度是较为缓慢的，不可避免会造成一定延时。因此，利用共享 Cache 将预取的数据送入主线程，会对程序的执行效率产生影响，也难以满足预取的及时性。

在上述帮助线程技术上改进，利用多核处理器中各个核心在线程之间的迁移是实现帮助线程预取的另一种方式^[37]。与数据传输具有延时的共享 Cache 相比，核间线程迁移技术则利用处理器各个核心之间，私有 Cache 的速度优势完成预取数据的访问，极大的提升了程序访存的速度与预取的效率。核间迁移的实现方式下，多核多线程处理器中一个核心执行主线程，其余内核并行执行剩余的线程用作预取。当程序执行一段时间之后，系统会对各个内核的执行内容重新分配。假设，内核 1 前一段时间执行预取线程，而内核 2 执行的是主线程。此时将这两个核心执行的内容互换，于是内核 1 已有后续执行所需的数据，因此主线程就可以利用存储在这个核心私有 Cache 中的数据信息加速自身执行的速度，从而实现访存效率的提升。

2.3 软硬件混合预取技术

预取指令依赖于编译时对应用程序的代码结构分析来实现预取，地址预测的准确性很理想，但指令开销与代码复杂度也随之增加。反观硬件预取，其无需单独的软件指令支持，主要依据访存过程中的历史信息来对程序的访存模式作出识别，但由于程序的局

部性往往只集中在某一区域，因此硬件预取的盲目性不容忽视。于是，考虑软硬件的优点，通过软件的方法来指导硬件电路更加准确的完成数据的预取便自然而然的成了研究人员的探索方向^[38]。

Zhang 和 Torrellas 合作设计一种面向于不规则存储单元的软硬件混合预取方法^[39]。该方法对内存中的数据添加一个标记位，当数据中的某一元素被访问时，那么该数据中的其它元素或该数据指针所指向的下一个数据将被会预取。此种访问方式在链式访存的存储形式中较为常见。该软硬件混合预取策略，利用软件方法对内存中数据的标记位进行复位。具体数据预取的实现，则由处理器中相应的硬件逻辑电路实现。

VanderWiel 和 Lijia 等人设计了一个由通用 CPU 构成的核外预取部件，该预取部件运行私有内部程序来为主处理器预取数据^[40]。预取而来的数据则通过共享的二级 Cache 完成传递，形成类似生物学中“生产者-消费者”的关系。与此同时，为了尽量降低总线带宽压力与减轻 Cache 污染，预取部件只有在预取的数据被使用后才会继续预取新的数据。主处理器则依靠软件的方法将生成的命令发往预取部件的内部寄存器，以此来达到指导预取部件的功能。

Wang 与 Burger 等人则提出一种引导性区域预取 GRP(Guided Region Prefetching)的软硬件混合预取方法^[41]。GRP 预取部件中的软件指令能够对程序访存过程中的访问规律进行分析，其余如预取的步长、预取深度、预取距离以及预取请求发出的时机等参数由编译器进行判断，硬件电路则主要用来完成具体的预取操作，从而最终实现软硬件搭配工作。

通过上述现有研究成果可以看出，软硬件混合预取大多设计较为复杂，因此在实际应用中有较高的门槛，难以广泛推广，目前市面上主流商用处理器中都没有对软硬件混合预取模式提供支持。

2.4 预取技术对比

软件方法会在程序中增加相应功能的设计代码来指导预取。硬件预取利用硬件电路对程序的访存模式识别来完成预取。软硬件混合策略则采用软件分析来协助硬件预取的思想。三种预取方式各有长短，都能在使用领域发挥效果，下表 2.2 简要总结了这三种预取策略的优缺点^[42]。

表 2.2 不同预取技术的优缺点

预取类型	优点	缺点
软件预取	程序行为特征分析充分，预 取准确度较高	增大了程序额外的代码量， 需要准确计算预取地址，预 取及时性较差
硬件预取	预取数据及时性较好，没有 额外的指令开销，	基于访存历史信息，没有充 分分析程序本身的特点
软硬件结合	兼顾软硬件预取的优点	需要软件与硬同时提供支 持，实现难度大，移植性差

2.5 本章小结

本章首先对处理器中不同访存操作完成同一个任务的耗时作出分析对比，证明了预取技术的加入能够有效的缩短整个执行过程的时钟周期，充分印证了预取策略对隐藏处理器访存延时的有效性。接着分别对三种预取方式的工作原理、实现方式以及现有的具体设计做出了较为详尽的叙述。其中，对于本文重点关注的硬件预取，更是通过丰富的示例，更深一步揭示了不同实现方式所依据的原理、改进方向以及适用的范围。最后，利用表格的方式清晰的总结出了三种预取机制的优缺点。通过本章的介绍，我们可以看出，硬件预取性能的优劣与程序本身局部性的好坏紧密相关，这也为下一章继续对内存中程序的访存行为特征的分析做出了铺垫。

第三章 内存中程序访存行为特征分析

现今以内存为主要核心的微处理器结构中，预取部件所能发挥效果的大小与内存本身的性能紧密相连。于是，对内存中程序运行时所表现出的访存行为特征的研究就显得十分必要。以下本章将通过处理器访问内存的地址方式、程序的局部性原理，以及内存中程序的访存模式并且结合示例进行分析，试图从中总结出程序访存过程中的一些特性，并基于这些特性为后续预取模块的设计提供方向。

3.1 处理器访问内存方式

现代存储系统中，主存储器一般由 M 个相同容量的存储单元数组所构成。其中，数组中所有元素都由独享的物理地址 PA(Physical Address)来标识，处理器则通过访问这些地址准确定位所要读取或写入的数据。例如，内存中数据块首字节编址为 1，下一个为 2，直到为所有 M 个字节完成编址。按照这种地址结构，处理器与主存之间的数据传输方式，应设计为基于 PA 的寻址系统。下图 3.1 给出了一个使用物理寻址的实例，假设该例中处理器译码访存指令后发出一个访存地址，从内存中读取 4 个字节的数据。当处理器执行这条指令时，会生成一个物理地址，通过系统中的地址总线发往内存。内存则依据地址取出数据，并通过数据总线返回给处理器中的寄存器，随后将会完成相应的运算操作。

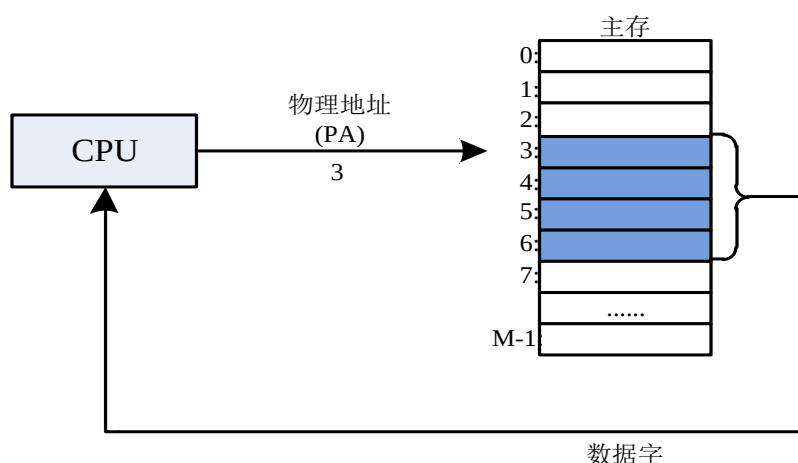


图 3.1 一个使用物理寻址的系统

物理寻址通常应用于稍早的计算机当中，目前诸如 DSP、嵌入式系统微控制器以及 Cray 超级计算机中物理寻址依旧发挥着作用。然而，为了提高主存的使用效率、更加有

效的对主存进行管理以及避免主存不命中的处罚延时，现代 CPU 架构往往采用一种被称为虚拟寻址的访问形式。虚拟寻址最早出现于 1960 年的 Atlas 计算机系统，其访问方式参见图 3.2 中的示例。

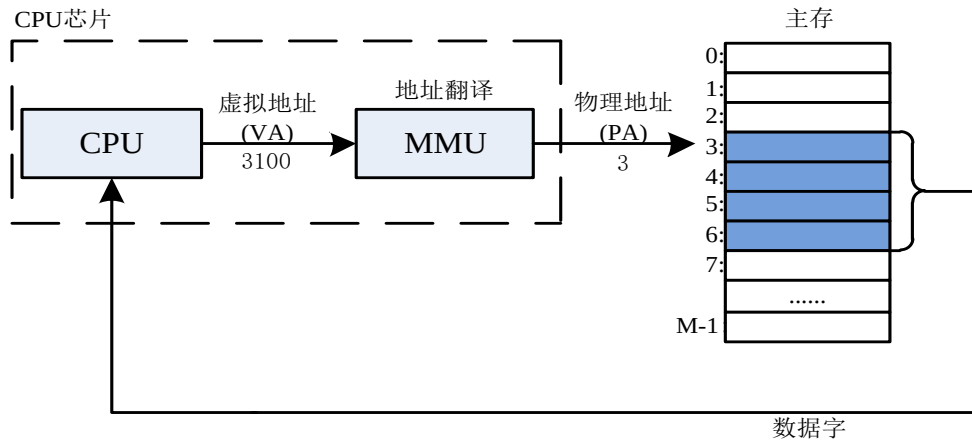


图 3.2 一个使用虚拟寻址的系统

虚拟寻址方式下，处理器读/写数据首先生成虚拟地址 VA(Virtual Address)，这个 VA 无法直接访问内存的字节，因此在将 VA 送入主存前，需要将其翻译为相应的 PA。目前，CPU 中的内存管理单元 MMU(Memory Management Unit)负责完成该项工作。每次 MMU 完成 VA 与 PA 之间的翻译时，都需要利用到主存中的页表结构，而页表则负责虚拟页与物理页联系起来。如下图 3.3 所示，与主存类似，页表的基本结构也是由条目组成的数组。每个条目中包含相应的地址信息。当条目中的有效位为高电平时，表示存储系统中的虚拟页已经送入内存中大小相等的物理页，此时条目中存储的内容则作为该页的物理页号 PPN(Physical Page Number)。

	有效位	物理页号
PTE 0	0	
	1	
	1	
	0	
	0	
PTE 5	1	

图 3.3 页表结构示意图

下图 3.4 展示了 MMU 如何通过页表将 VA 翻译为 PA 的具体过程。CPU 内部的页表基址寄存器用于定位页表，完整的 n 位 VA 主要分为两段： p 位的虚拟页偏移量

VPO(Virtual Page Offset)与(n-p)位的虚拟页号 VPN(Virtual Page Number)。翻译操作执行时, MMU 首先利用 VPN 寻找与其对应的页表条目。例如, VPN0 对应页表中第一个条目, VPN1 对应第二个, 余下按此一一对应。当 VPN 对应的条目中有效位为高电平时, MMU 就会将条目中所存储的 PPN 与 VA 中剩下的 VPO 拼接起来, 最终得到访问内存所需要的实际地址 PA。在当前计算机存储结构中, 物理页与虚拟页的容量大小默认一致(内存类似于磁盘的缓存, 内存与磁盘交换的数据块在内存中称为物理页, 在磁盘中则叫做虚拟页), 因此 PA 中的 PPO 与 VA 中的 VPO 位数相同, 无需映射。当 MMU 完成地址翻译的任务后, 就将物理地址送往高速缓存/内存中, 高速缓存/内存则返回地址所指向的数据给处理器。

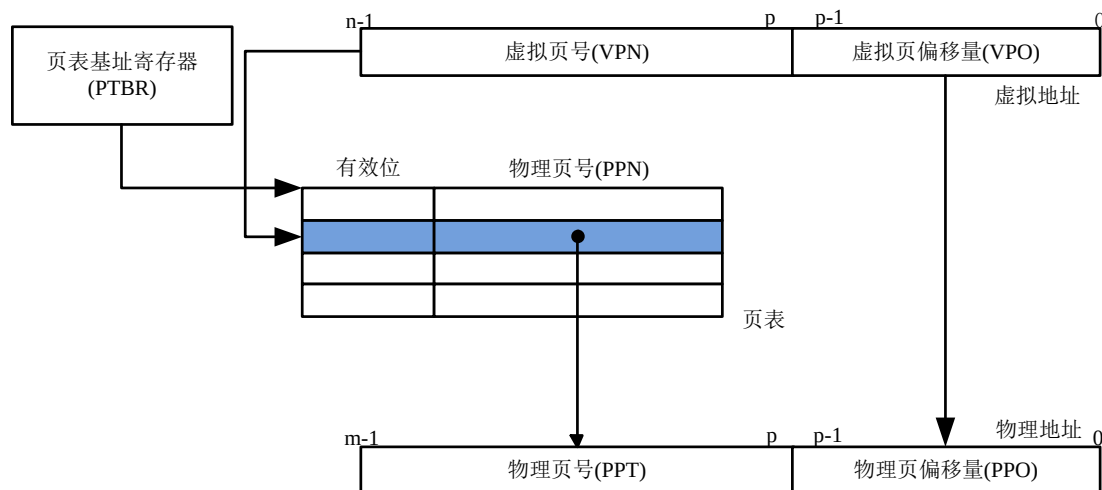


图 3.4 使用页表的地址翻译过程

通过上述对处理器访问内存数据行为方式的分析可知: 处理器访问内存/缓存首先会生成虚拟地址, 随后通过专用硬件 MMU 再翻译为实际的物理地址。于是, 我们在后续预取部件的设计当中, 对程序执行过程中访存模式的识别以及预取地址的计算都可以在虚拟地址下进行, 而无需考虑后续的地址翻译过程。因为, 虚拟地址与物理地址之间通过页表形成映射关系, 这种关系不会对预取的正确性造成影响。

3.2 程序局部性原理

预取部件的设计应该充分理解程序的局部性原理, 在局部性越好的程序中, 预取部件所能检测到可预测的访存模式的概论往往也就越大, 从而预取的效率也会越高。

局部性可理解为: 程序运行时, 与内存中当前被访问的元素位置相近的元素或当前

被访问元素的本身，在后续执行时很可能会被再次访问。此种访存特征表明，程序运行时处理器访问内存地址在一定时间范围内是呈现聚集趋势并相对连续的，而不是随机分散。局部性可分为时间/空间局部性两种。前者指的是：内存程序中正在被处理的数据或指令，会有很大几率被接下来发出的几个访存操作再次命中。后一种则为：处理器下一个访存操作所需要的数据或指令，在地址上很大概率与现在正在被操作的数据或指令相邻。

通过对众多应用程序访存行为特征的分析与统计，在给定一段有限的时间范围内，处理器访问程序中指令与数据的内存地址大概率会分布在整个程序内存地址空间的某一段区域。这是因为程序中，代码结构会包含着循环与遍历片段，这些代码段在程序运行期间会被执行多次，于是，具有此种代码结构特征的程序在运行时就会体现出上述介绍的局部性。

程序运行时潜在的局部规律性，为预取策略的实现提供了可行性方向。如果处理器连续访问数据的地址大多落在某一较小的地址空间段内，那么这些访存操作之间就可能存在某种有规律的地址差或重复出现的地址序列等访存模式。预取部件所要做的就是识别出这种可能存在的地址之间的规律性，并预先获得这些数据。

3.3 内存中应用程序的访存模式

介绍完程序局部性后，下面将进一步对程序访存模式进行分析。访存模式即为处理器发出读写指令，对内存中程序进行访问所体现出来的一种具有可循规律的操作方式。通过对大量典型程序运行过程的分析，研究人员发现程序常常表现出有规则、地址相关以及随机性等多种内存访问模式^[43]。程序中内存访问模式与预取部件的设计关系紧密，目前，大部分所提出的预取算法都是针对于某一个固定的访存模式，并且不适用于其余的访存模式中。因此，基于程序中最常出现的访存模式并设计合适的预取算法，就能极大的提升预取部件的覆盖率与准确率。

3.3.1 基本访存模式

按照连续访存请求访问内存地址之间的关系，系统中程序基本访存模式主要可分为：友好访存模式、数据流访存模式、链式访存模式以及随机访存模式这四大类^[44]。

友好访存模式指的是在一定时间范围内数据呈现被多次访问的特征，程序中此种模

式具有良好的时间局部性。下图 3.5 (a)给出了一种典型的友好访存模式访问序列，其中 N 为地址序列重复访问的次数。这种具有重复访问规律的地地址序列可以通过硬件的方法进行检测，一旦发现图中所示的访存序列时，预取部件就会适时发出对后续数据的预取请求。上一章中所提到的 Markov 预取就是友好访存模式下的一个应用实例。

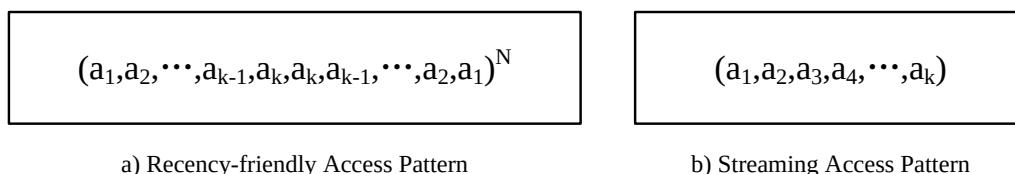


图 3.5 访存模式示例

数据流访存模式现今主要以循环数组等形式广泛存在于大型科学计算、工程应用以及大多数多媒体应用程序中^[45]。该访存模式下，处理器在一定范围内对图 3.4(b)中的数据 $a_1, a_2, \dots, a_k$ 的访问具有地址递增或递减的可识别规律性。数据流访存模式按照访问相邻地址之间的步长大小又可进一步分为顺序数据流与跨步数据流：顺序数据流即为访问地址间隔不超过一个 Cache 行大小的数据流，并且数据流中起始数据块与终止数据块之间的每一个 Cache 块都会被依次访问；访问地址步长间隔大于 1 个 Cache 行的数据流则被称为跨步数据流，一般多个连续访存请求中 3 个访存地址之间两两地址差相等即可认为构成跨步数据流。数据流访问方向也由正向与逆向两种：正向访问即为按照 Cache 行地址递增的访问方式；逆向访问则为地址递减，例如由数组末尾向开头进行遍历。数据流访存模式是程序中最普遍的内存访问方式，本文也将基于该访存模式来进行预取部件的设计，并希望通过合理的数据流识别算法，最大程度隐藏处理器访存延时。

链式访存模式是一种具有地址相关性的内存访问模式，其主要存在于链表(Linked Data Structure)、 n 元树以及其它图形数据结构中。此种访存模式下指令之间存在着一定的关联性，这种地址相关性可能是一对一或一对多的关系。如下图 3.6 所示，在链式访存下，链表数据结构中的结点与结点之间一般需要通过指针链接，每个结点中存有数据以及指向下一结点的地址(头结点中没有数据，只有指向下一结点的地址；尾节点中地址为空，表示链表到此结束)，由于结点之间的地址不存在同步性，下一结点的地址只有在当前结点被访问之后才会变得可见，这往往会造成指针追逐的现象，进而引起大量的指令访存失效。链式访存模式地址相关性依赖较大，程序中缺乏局部性，这对于预取部件的设计提出了较高的要求，目前也有一些学者针对链式访问的预取策略作出了一定的研

究^[46,47]。

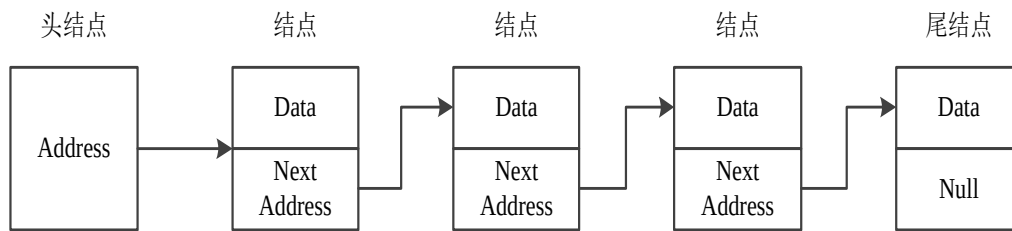


图 3.6 链式访存模式

在实际的应用程序中，除了有规则的访存模式之外，还存在着大量无规则的随机访问，此种访存模式往往是不可预测并且对 Cache 与主存的性能是有害的。随机访存模式并不是指对内存真正的随机访问，而是程序执行时对地址的访问通常是没有任何可循规律。数据结构与动态内存分配算法中一般会存在随机访问的特征：例如，哈希表一般被设计为在整个表中随机分布元素以避免冲突，因此，对表中的元素的查找是以随机访存模式进行的；其次，树状结构中对树中不同路径的遍历也会导致无规则访问。内存中随机访存模式的程序既不存在局部性又不依赖地址相关性，因此很少有预取策略能够发挥作用。

3.3.2 数据流访存模式实例

研究处理器访问内存中程序访存模式最典型的例子是矩阵(包括一维数组与多维数组)。考虑下图 3.7(a)中的一个简单函数，它实现的功能为对一维数组中的每一个元素进行累计求和。首先我们对这个程序的访存模式进行分析。在这个例子中，函数中用于暂存结果的参数 `sum` 在整个求和的运算当中反复多次被执行，于是它满足了时间局部性。正如从图 3.7(b)中看到的那样，数组 `v` 中的元素是一个接一个按序读取的，为了方便，我们假设数组是从地址为 0 开始访问，并且元素之间的地址差为 4。因此，数组 `v` 在执行过程中满足空间局部性。

对于 `sunvec` 这样以顺序访问数组中的每一个元素的函数，其对内存的访问即为顺序数据访存模式(相对元素的大小，其访问地址之间的间隔为 1)。如果程序每隔 `k` 个元素进行一次访问，则称为步长为 `k` 的跨步数据流访存模式。数组中这些数据流访存模式的存在为预取提供了前提。一旦预取部件识别出这种数据流访存模式，那么就能够实现对该数组后续数据的预取，有效的提升了缓存中数据的命中率，从而实现隐藏处理器访存延时的目的。

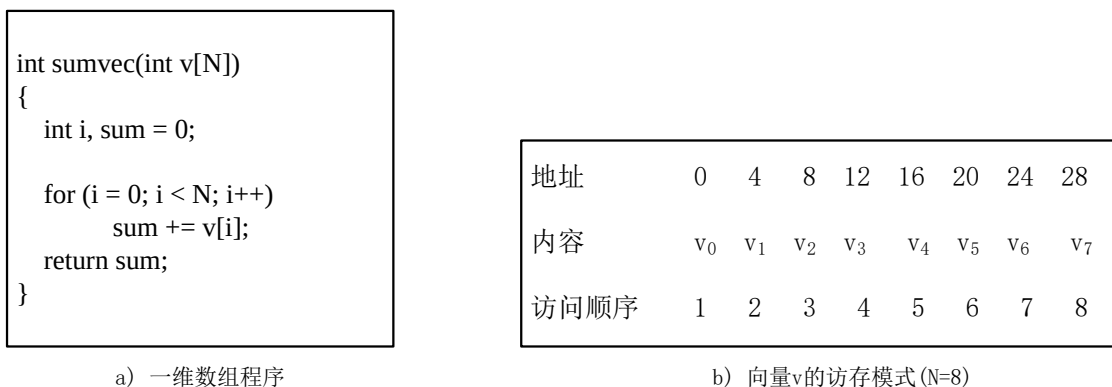


图 3.7 顺序数据流访存模式

对于多维数组的程序来说，其执行的效率也与其被访问的方式紧密相关。多维数组在连续的内存当中一般以行格式进行存储(按照一行一行的方式存入内存当中)。下图 3.8(a)中的函数 `sumarrayrows`，它的功能是对二维数组中的元素遍历求和。程序运行时，代码中两个 `for` 循环依据行优先访问规则(row major order)对二维数组执行相应操作。具体方式为，内外层 `for` 首先循环访问第一行数据，接下来访问下一行，依此类推，形成数据流。图 3.8(b)给出了该数组从主存送入 Cache 后被读取时的访问情况。以第一轮迭代为例，访存请求命中 Cache 行，随后的访存操作依旧会命中同一 Cache 行中的数据元素，并且一次迭代访问过的 Cache 行随后也不会再次访问。此种访问方式大大降低了 Cache Miss 的几率，并且使得每一 Cache 行中的数据利用率得到了最大化。如果该数组程序执行过程中，处理器发出的访存请求命中的数据没有从主存送入 Cache 行中，那么就会带来额外访问主存的延时。从预取的角度来看，此种数据流访问方式易被识别、访存效率高且获益大。预取部件检测出数组访问时的数据流后，发出一次预取操作可取回 Cache 行大小的数据，并且预取回来的数据在随后的求和过程中很快就会被执行，从而很好的避免了多次读取内存所产生的开销。

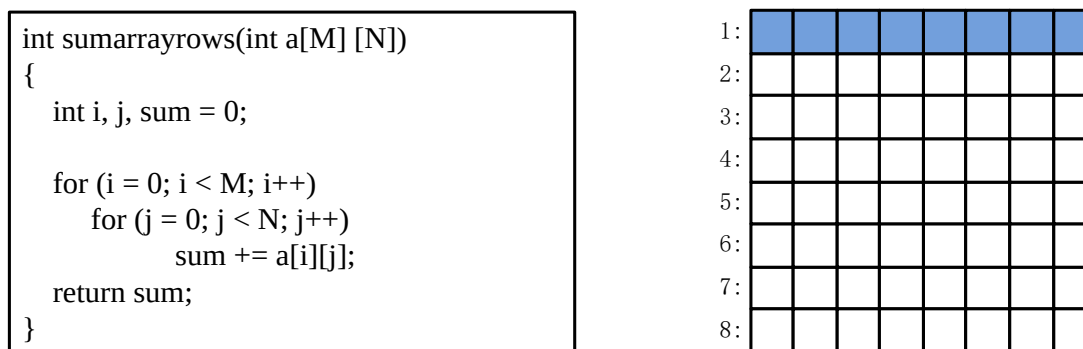


图 3.8 a)行优先访问的二维数组(M=8,N=8); b)Cache 中二维数组循环访问方式

3.4 本章小结

本章主要关注的是内存中程序运行时所表现出的访存行为特征。首先，文中分析了虚拟寻址方式下页表完成地址转换的翻译流程与工作原理。接着，讨论了程序访问的局部性原理，为预取部件设计的可性能提供了理论的支撑。最后，重点介绍了程序中主要存在的几种访存模式，总结了不同访存模式下访存地址所体现出的规律性，并对不同访存模式下预取部件设计的可行性进行了说明。其中，对于本文关注的数据流访存模式来说，本章也结合了计算与工程领域中常见的数组程序代码，讨论了预取技术预期可能发挥的效果。通过本章的分析，下文中预取部件的设计可以选择在虚拟地址的基础上，分别对程序中常见的顺序数据流与跨步数据流两种访存模式制定合理的检测算法，力求同时保证预取部件的准确性与高效性。

第四章 基于数据流访存模式的硬件预取实现

4.1 预取地址

通过上文处理器访问内存的地址方式分析可知,层次型存储结构中 CPU 访问 Cache 时,首先会生成虚拟地址,随后再通过地址翻译计算出实际访问内存中数据的地址。本文所设计的基于数据流访存模式的预取部件中,对访存地址的识别、更新以及预取地址的计算都是在虚拟地址的基础上进行的,以便较早获得流水线上访存信息。

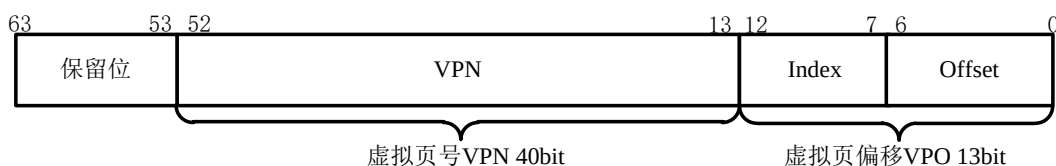


图 4.1 64 位访存地址结构示意图

上图 4.1 给出了本文所采用的国产某型号自主指令集处理器的 64bit 访存地址结构示意图。该 64 位虚拟地址主要由保留位、VPN 以及 VPO 三个地址字段组成。地址字段 [52:13] 为虚拟页号 VPN, 翻译时会定位到页表中相应条目并取出物理页号 PPN, 来作为该页的索引。本文中处理器一级数据 Cache 为 8 路组相联结构, 因此在实际访问数据 Cache 时, 物理页号 PPN 会与可能出现的 8 个不同的物理页号来进行匹配, 判断所访问的数据是否来自该 PPN 所索引的物理页中, 从而确定访存请求是否命中 Cache。此外, 本文所设计的硬件预取部件时不允许跨物理页获取数据的, 即访存地址中的 VPN 与预取地址的 VPN 是一致的 (PPN 也相同), 这也降低了预取部件获取数据的复杂度。

13 位的虚拟页偏移字段 VPO 定位数据的实际位置, 从上一章的介绍中, 我们得知虚拟页偏移与物理页偏移位数相同无需映射, 这是因为当前存储系统中虚拟页与物理页大小相同。地址 [12:7] 位具体表示数据在内存中物理页的哪一块, 实际访问 Cache 时会用来索引访问的数据在 Cache 的哪一组中, 随后再通过 PPN 判断该组中是否有数据命中。该 6 位地址与 VPN 组成的地址字段 [52:13] 可以用来代表翻译后的 Cache 行地址, 并作为后续预取部件检测程序中数据流访存模式的地址识别信息。地址字段 [6:0] 位则表示所访问数据在 Cache 行内的字节偏移, 用于定位到所访问内容的首字节地址, 从而实际取出或写入规定字长的数据。本文中每次预取数据的大小为一个 Cache 行, 因此该 7 位地址字段在数据流识别过程中保持不变, 不对预取的正确性构成影响。

4.2 预取部件的位置

硬件预取部件的设计与系统中处理器访存路径紧密相关，因此在预取部件实现前，分析访存流水线，找出预取部件适合放置预取部件的位置就显得十分必要。下图 4.2 给出了本文中某型号国产处理器平台的访存数据通路。指令发射部件接受来自处理器内部的访存指令，并发往整数执行部件形成访存请求。随后访存请求进入 Cache 控制部件中的访存流水线，再经过地址翻译、数据 Cache 标记管理单元等操作，最终判断访存请求是否命中一级数据 Cache(DCache)。如果访存请求命中 DCache，对于读类型请求来说，所需要的数据会送入处理器内部的逻辑运算单元，随后处理器执行后续操作；对于写类型请求，数据将会从处理器内部寄存器中按照地址写入 DCache 中的具体一行中。如果访存请求没有命中 DCache，那么该请求会被送入地址失效缓冲 MAF(Miss Address File)。MAF 作为 DCache 与二级 Cache 之间的悬挂缓冲，会缓存发往二级 Cache 且暂未收到响应的访存失效请求，当二级 Cache 向 DCache 送入处理器所需数据时，MAF 中相应信息也会立即删除。

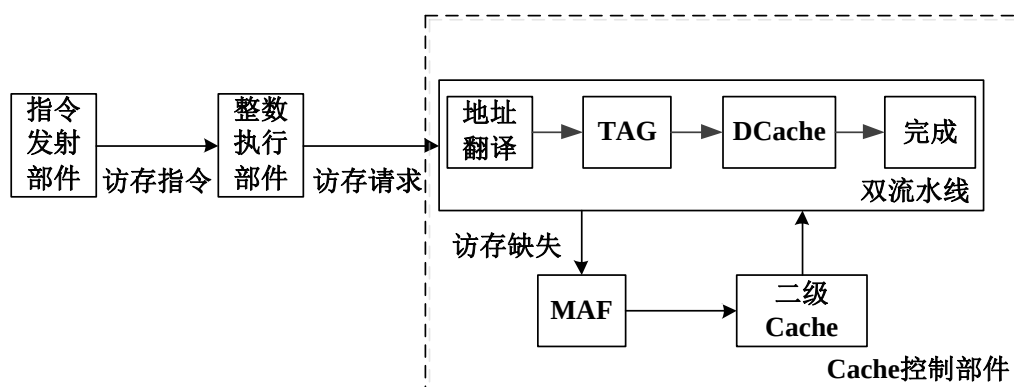


图 4.2 国产处理器访存流水线通路

通过上述对处理器访存通路的分析，本文可选择在 Cache 控制部件中完成硬件预取部件的实现，其具体位置如下图 4.3 所示。预取部件在访存流水线执行的同时，并行记录访存流水线上的访存地址信息，再通过相应的预取识别策略完成程序中数据流访存模式的识别，生成硬件预取请求。预取部件具体目标功能如下：1、记录访存流水线上的访存地址信息；2、识别程序中的数据流访存模式，计算预取地址，并将生成的预取请求先行发往预取缓冲中暂存；3、将预取缓冲中的预取请求发往各级 Cache 当中，其中一级 Cache 预取请求，还会发往访存流水线进行重试，从而判断该预取地址的数据是否已经在 DCache 中。

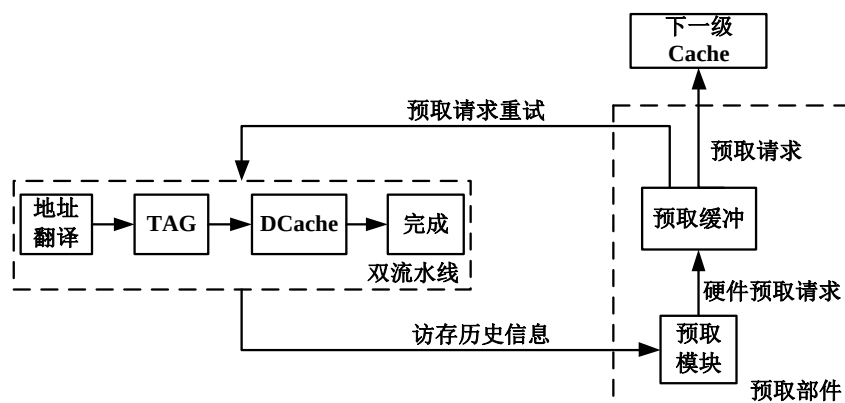


图 4.3 预取部件的位置

4.3 预取部件的结构与组成

如图 4.4 所示，预取部件又由预取控制模块(PFH_Ctrl)、数据流预取条目模块(Stream_Ctrl)、跨步数据流检测模块(Stride_Ctrl)以及预取缓冲条目(PFH_Buff)组成。其中，预取控制模块记录来自访存流水线上的访存信息，并控制预取部件的执行；数据流预取条目模块用来对顺序数据流进行识别，并存储识别出的数据流；跨步数据流检测模块主要负责跨步数据流的识别；预取缓冲条目则用于存储生成的预取请求。

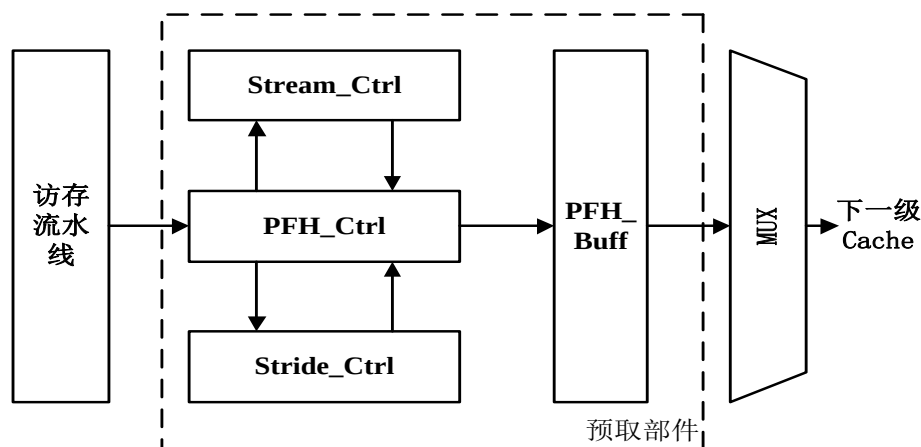


图 4.4 预取部件结构

预取部件完成一次数据流识别操作的大致流程如下图 4.5 所示，当访存请求有效时，预取控制模块接受来自访存流水线上的访存信息，并对访存信息的相应参数，如访问类型、访问的 Cache 行地址等信息进行分析判断，随后访存地址被送入数据流预取条目模块进行顺序数据流的识别。如果识别到顺序数据流访存模式，预取控制模块会计算出对各级 Cache 的预取地址，生成预取请求并发往预取缓存条目等待后续操作。如果访存请

求没有命中预取条目，那么该访存请求会被送入跨步数据流检测模块，进行跨步数据流的识别。一旦跨步数据流被成功识别，那么预取控制模块也会生成相应的预取请求，并发往预取缓冲条目中，否则本次访存操作就不会生成预取请求。

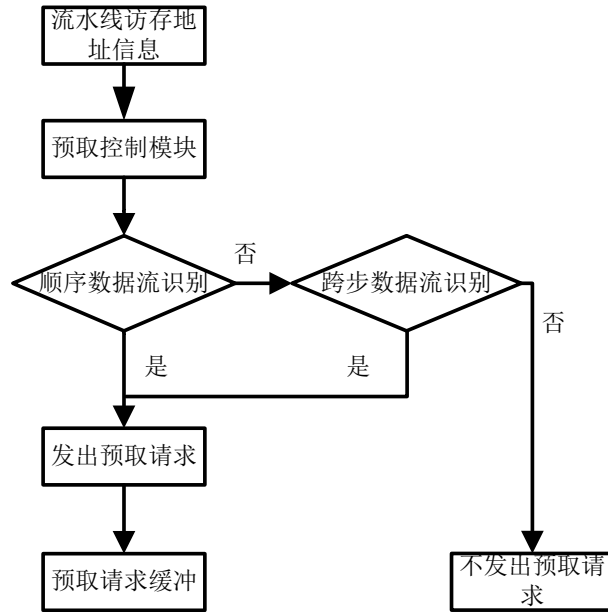


图 4.5 预取请求生成流程图

4.3.1 预取控制模块

访存流水线中的请求信息通过预取控制模块(PFH_Ctrl)中的输入端口进入预取部件，同时整个预取部件的有序执行也需要在预取控制模块的指示下进行。预取控制模块的主要预期功能为：1、并行接受访存流水线上的访存请求信息；2、识别访存请求的类型；3、判断访存请求是否命中预取条目，并根据结果执行后续的相应操作；4、当识别出数据流访存模式后，计算预取地址，并将生成的对各级 Cache 的预取请求发往预取缓冲当中。

预取控制模块是整个预取部件中最关键、最复杂的逻辑单元，它不仅需要接受外部访存请求信息，同时还需要对数据流识别过程中生成的命令作出响应，准确指导预取部件中各个子模块之间的交互。以下将对预取控制模块部分信号接口的具体功能进行进一步的介绍。

表 4.1 预取控制模块主要端口信息

端口信号名	具体含义
i_PipeReqValid	访存请求有效位

表 4.1 (续)

端口信号名	具体含义
i_PipeReqAddr	当前访问地址
i_PipeCMD	访存请求类型
o_ReqRCnt	预取请求的预取距离
o_PfhReq	生成的预取请求

预取控制模块的主要端口信号如上表 4.1 所示，在输入信号中，i_PipeReqValid 表示当前信息有效位，低电平有效。信号 i_PipeAddr 为当前指令访存的虚拟地址，该地址信息进入预取部件后，会根据信号类型判断的结果执行后续操作。信号 i_PipeCMD 为判断访存请求类型的端口，如果当前访存请求为普通读/写请求，预取控制模块会将该访存请求地址送入数据流预取条目模块，来判断该请求是否命中数据流预取条目，一旦请求命中条目时，预取控制模块会发出相应的命令控制预取条目的更新，如果请求没有命中预取条目，预取控制模块则会为其分配一个新的预取条目。如果当前请求为预取请求或其它请求类型，则无需进行后续操作。

在输出信号中，o_ReqCnt 为当前预取请求的预取距离(Prefetch Distance)。预取距离即为预取数据的地址与当前访存地址之间的地址差。在本设计中，对于多级缓存的预取距离，规定设置为 1、4、10。端口 o_PfhReq 为模块发出的预取请求，预取条目识别出数据流访存模式后，向预取控制模块发出相应信号，预取控制模块对信号进行处理，依据预取距离等参数设置，最终计算出多级 Cache 预取的地址，并将预取请求发往预取缓冲进行下一步的处理。例如，对于一个被识别出的顺序数据流，其当前所访问的 Cache 行地址为 A，那么所生成的一级/二级/三级 Cache 的预取地址分别为 A+1、A+4 以及 A+10。

4.3.2 数据流预取条目模块

数据流条目检测模块(Stream_Ctrl)主要用于数据流访存模式中顺序数据流的识别与储存，并允许已经检测到的跨步数据流插空分配条目。数据流预取条目中主要记录访存请求的地址信息、当前访问数据流的方向以及预取条目被访存请求命中的次数等信息。本文共设有 16 个预取条目，能同时实现对多条数据流的检测，最大限度的提升了预取部件的效率。每个预取条目中包含的部分寄存器信息具体见下表 4.2 所示。

表 4.2 预取条目部分寄存器端口信息

寄存器信号名	具体含义
r_Valid	当前条目有效位，高电平表示条目有效
r_Addr	当前访存操作的地址信息
r_CCnt	预取条目的命中次数
r_MCnt	预取条目命中次数的最大值
r_Dir	当前访问的数据流方向
r_NextEA	按照当前方向下一个将被访问的地址
r_ShadeA	当前方向反方向下一个将被访问的地址
r_FuzzyEA	按当前方向下一个将被访问的地址

信号 **r_Valid** 表示当前预取条目的有效位，数据流识别时，只有在此信号为高电平时预取条目才有效，否则预取条目无效且不具备检测数据流的功能。端口 **r_Addr** 代表由预取控制模块送来的访存请求所访问的 Cache 行地址(虚拟地址中的[52:7]地址字段)，其功能用于判断访存请求是否命中预取条目。如果访存请求不命中，则分配新的预取条目，同时预取控制模块还会将该访存请求的地址信息送入跨步数据流检测条目；如果命中预取条目，则对预取条目中的寄存器信息进行更新。

信号 **r_CCnt** 表示预取条目被占用后，接下来的访存请求访问该预取条目的情况。当后续流水线上的访存请求每次命中该预取条目一次，**r_CCnt** 的数值就会增加 1，直至达到命中次数的最大值 **r_MCnt**。一旦预取条目的命中数来到规定的阈值 **r_MCnt** 时，预取部件每次发出请求的个数，即预取深度(Prefetch Depth)也达到最大值。预取深度简单来说就是每次发出预取请求的个数，例如，对于一个顺序数据流其步长为 0x128，程序最近一次访存地址为 0x64，当预取深度为 3 也就是发出 3 个预取请求，则对应的预取数据的地址为 $0x128i+0x64(i=1、2、3)$ 。

达到最大预取深度后如果后续的访存请求继续命中预取条目，预取深度也不会发生改变，这样主要是为了避免过大的预取深度会带来额外无效的预取数据，防止造成 Cache 污染。本设计中当访存请求连续命中三次预取条目(包括初次分配条目)，即可以看作识

别出一条顺序数据流。当模块识别出顺序数据流后，随后访存请求每命中该条目 2 次，预取深度加 1，预取深度最大不超过 4。

信号 `r_Dir` 表示数据流访问的方向，低电平 0 表示按照访问的 Cache 行地址递增，高电平 1 则表示递减访问 Cache 行地址。`r_NextEA`、`r_ShadEA` 以及 `r_FuzzyEA` 这三个条目寄存器信号，分别表示按照当前方向访问的 Cache 行地址的下一个地址、按当前数据流方向相反的下一个地址以及按照当前方向访问的下下一个地址。这三个信号均用于对后续访存请求地址的猜测，并作为后续访存请求是否命中预取条目，以及是否形成数据流的判断标准。在本设计中 `r_ShadEA` 与 `r_FuzzyEA` 共一个地址寄存器，在寄存器中分别设置两个有效位 `r_ShadValid` 与 `r_FuzzyValid`，在对数据流进行检测时 `r_ShadValid` 设置为高电平，即 `r_ShadEA` 有效；而当数据流已经建立的前提下，对数据流进行追踪与更新时 `r_FuzzyValid` 则变为高电平，此时寄存器位 `r_FuzzyEA` 替代 `r_ShadEA` 变为有效。

4.3.3 跨步数据流检测模块

除了单位 Cache 行的顺序数据流外，程序中往往会存在着许多大步长的访存操作，这种访问在数组或矩阵等较为密集的大型科学计算中存在几率较大。例如，访问多维数组非相邻 Cache 行数据就是跨步数据流访存模式的一个实例。跨步访问的地址差易于计算，同时还存在数据流的相关特性。于是，为了使所设计的预取部件更加高效，本文在对顺序数据流检测的同时也单独设置了跨步检测的模块，这也最大限度的避免了不同步长的数据流在检测时可能会出现的干扰。

跨步数据流检测模块(`Stride_Ctrl`)主要功能为识别步长大于 1 个 Cache 行大小的数据流，模块中共设有 6 个这样的条目来完成目标，每个条目中部分寄存器信息如下表 4.3 所示。

表 4.3 跨步检测模块部分寄存器端口

寄存器信号	含义
<code>r_Addr</code>	当前访存操作地址信息
<code>r_Stride</code>	访存请求与分配条目的地址差
<code>r_StrideValid</code>	地址差有效
<code>r_StrideEn</code>	当前地址差即为跨步数据流

信号 r_Addr 表示当前访存请求的地址信息，当访存请求没有命中数据流预取条目时，该访存请求在分配预取条目之外，还会被送入跨步数据流检测模块，进行跨步数据流访存模式的识别。信号 r_Stride 指的是跨步数据流识别的过程中，条目中先后访存请求 Cache 行地址之间的差值。信号 $r_StrideValid$ 为访存请求之间 Cache 行地址差是否有效的标志位，只有在 $r_StrideValid$ 为高电平时，请求之间的地址差才有意义。 $r_StrideEn$ 则代表模块识别出跨步数据流时的信号位。本文规定，当最近一段时间被送入跨步数据流检测模块的 6 个访存请求(不要求连续)中有三个访存地址之间构成了固定的 Cache 行地址差，此时即为成功识别出跨步数据流并触发预取。这时 $r_StrideEn$ 变为高电平，跨步数据流检测模块向预取控制模块发出命令。预取控制模块收到跨步数据流检测模块的信号后，会向数据流预取条目发出控制信息，指导识别出的跨步数据流插空写入预取条目中，并控制该数据流后续的更新。

4.3.4 预取请求缓冲

预取缓冲条目(PFH_Buff)主要用来存储预取部件所生成的预取请求，如图 4.6 所示，预取缓冲采用先入先出(FIFO)的存储结构设计。缓冲条目设置有三个写端口与一个读端口，条目采用读写指针的跳转以循环队列的方式来对预取条目中预取请求的进行读写控制。具体的算法为：在一个时钟周期内预取缓冲处理 4 条预取请求的写入，其中一级/二级/三级预取请求，分别从图中各自写入的端口仲裁进入预取缓冲。各个预取请求写入预取缓冲的优先级规定如下：一级预取请求最高，二级预取请求次之，三级预取请求最低。当预取缓冲条目满时，此时预取请求无法继续写入条目中，直接被丢弃，只有等到缓冲中有空闲条目时请求才能再次进入。

进入预取缓冲的预取请求需等待仲裁，然后才从读端口上流水线进行后续的相应处理。为了不影响正常指令执行，本文规定预取请求优先级低于普通访存请求。在流水性空闲时，预取请求仲裁上访存流水线，当发生一级预取请求命中一级数据 Cache，表明所要预取的数据已经在 Cache 当中，根据多级 Cache 之间的包含关系，此数据也会在层次更低二级/三级缓存当中。这种情况下，该预取请求被视为无效，不会继续执行；反之该请求将继续送入下一级 Cache 中。同理，二级/三级预取请求也按照上述原理判断是否发往下一级 Cache 或主存中。

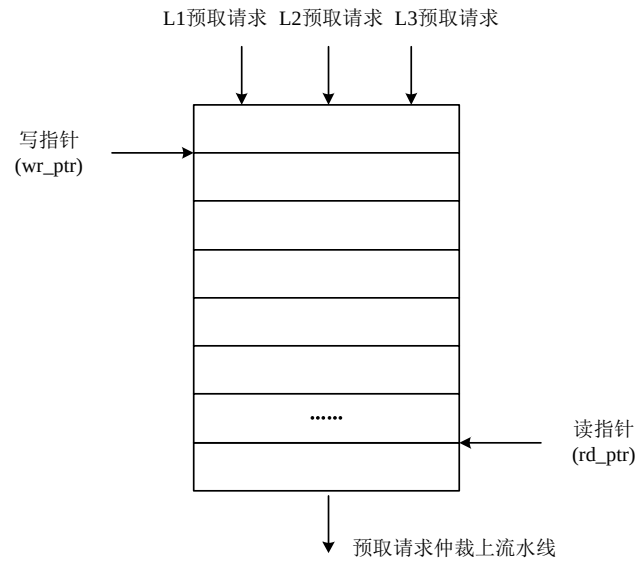


图 4.6 预取缓冲结构图

4.4 数据流访存模式的识别示例

4.4.1 顺序数据流的识别

预取部件中设置了若干个预取条目，用于进行顺序数据流的识别与更新。基本原理为：如果检测到连续 3 个访存操作访问的 Cache 地址构成一个连续递增或者递减的访存序列(新的访存请求初次分配预取条目，后续访存请求连续命中该预取条目两次)，就会被识别为一个顺序数据流。接下来以几个典型的顺序数据流为例，具体说明数据流的识别过程，其中 $A/A+1$ 等均为访存指令访问 Cache 的行地址， $A+1$ 与 A 之间为一个 Cache 行地址差。

1) 顺序递增数据流 A 、 $A+1$ 、 $A+2$;

如下表 4.4 所示，当来自流水线的访存操作访问的 Cache 地址为 A 时，此时分配预取条目，此时猜测数据流的访问递增向即寄存器 r_Dir 置为 0， r_NextEA 中写入下一个访存请求可能访问的 Cache 行地址 $A+1$ ，同时寄存器中 $r_ShadValid$ 位变为有效， r_ShadEA 中写入相反方向可能访问的下一个顺序 Cache 行地址 $A-1$ 。随后当来自访存流水线的第二个访存请求，访问的 Cache 地址为 $A+1$ 时。此时访存请求命中预取条目 r_NextEA 寄存器中记录的地址 $A+1$ ，记为第一次命中预取条目。接着 r_NextEA 中的写入的地址信息变为按顺序的下一个访存地址为 $A+2$ ， r_ShadEA 中则保持原本写入的地址信息不变。当第三个访存请求继续命中此预取条目，即访问的 Cache 行地址为 $A+2$ 时，

记为访存请求第二次命中预取条目，这时即为检测到一条顺序数据流，其访问 Cache 地址序列为 $A \rightarrow A+1 \rightarrow A+2$ 。于是预取部件便生成对后续多级 Cache 的预取请求，并发往预取缓冲中。

顺序递减数据流 $A \rightarrow A-1 \rightarrow A-2$ 的识别原理亦是如此，此时寄存器 r_Dir 应当置为 1 表示猜测的方向与当前访存请求相反。

表 4.4 顺序递增数据流检测示例

当前访存地址	r_NextEA	r_ShadEA	猜测方向	说明
A	A+1	A-1	顺序递增	访存地址 A 初次分配条目
A+1	A+2	A-1	顺序递增	命中条目中的 r_NextEA , 方向猜测正确, r_ShadEA 保持不变
A+2	A+3	无效	顺序递增	命中条目中的 r_NextEA , 方向猜测正确, r_ShadEA 变为无效, 识别出递增的顺序数据流访存模式

2) 带干扰项的顺序递增数据流 A 、 $A-1$ 、 $A+1$ 、 $A+2$ 、 $A+3$

如下表 4.5 所示，第一个访存请求访问的 Cache 行地址为 A，首先分配预取条目，猜测访问方向为正向 r_Dir 置为 0， r_NextEA 中写入地址 A+1，寄存器 $r_ShadValid$ 置为有效， r_ShadEA 写入地址 A-1。第二个访存请求访问地址 A-1，命中 r_ShadEA 中的猜测的地址信息，此时 r_Dir 寄存器置为 1 猜测方向变为相反的方向， r_NextEA 中的地址信息变为 A-2，这时 r_ShadEA 中写入 r_NextEA 之前记录的地址信息 A+1。接下来访问的 Cache 地址为 A+1， r_Dir 寄存器再次变为 0 方向再一次猜测为正向， r_NextEA 中内容变为 A+2， r_ShadEA 变为 A-2。后续的访存操作则按照上述递增数据流检测的方法进行工作，最终识别出带有干扰项的 $A \rightarrow A-1 \rightarrow A+1 \rightarrow A+2 \rightarrow A+3$ 访存序列。

其余带有干扰项的顺序递增/递减数据流，都可按照上述的数据流识别策略进行检测。

表 4.5 带干扰项的顺序递增数据流识别示例

当前访存地址	r_NextEA	r_ShadeA	猜测方向	说明
A	A+1	A-1	顺序递增	初次分配预取条目
A-1	A-2	A+1	顺序递减	命中条目中的 r_ShadeA, 方向猜测错误, 取反
A+1	A+2	A-2	顺序递增	命中条目中的 r_ShadeA, 方向猜测错误, 取反
A+2	A+3	A-2	顺序递增	命中条目中的 r_NextEA, 方向猜测正确, r_ShadeA 保持不变
A+3	A+4	无效	顺序递增	识别出递增的顺序数据流访存模式

4.4.2 跨步数据流的识别

假设流水线每隔若干时钟周期产生一个访存请求, 预取控制模块接收访存请求后首先进行顺序数据流的识别, 如果访存请求没有命中预取条目, 预取控制模块则将这些访存请求发往跨步检测模块进行跨步数据流的识别。

如下图 4.7 所示, 假设处理器连续访存请求访问的 Cache 地址依次为 A、B、C、D、E、F, 跨步识别条目会记录当前访存地址与之前每一个访存地址之间的地址差, 一旦预取部件识别出跨步数据流访存模式, 此时跨步检测模块会向预取控制模块发出请求, 申请预取条目, 用于后续数据流的追踪。在申请预取条目期间, 不再识别新的跨步数据流。在进行跨步数据流的识别过程中, 可能检测到多个跨步数据流。例如, 同时发现地址差 $F-E = E-D$ 和 $F-B = B-A$, 即构成地址序列 $D \rightarrow E \rightarrow F$ 和 $A \rightarrow B \rightarrow F$ 这样两个跨步数据流。此时, 优先选择与低位条目记录地址构成的跨步数据流申请预取条目, 即选择 $A \rightarrow B \rightarrow F$ 跨步数据流。

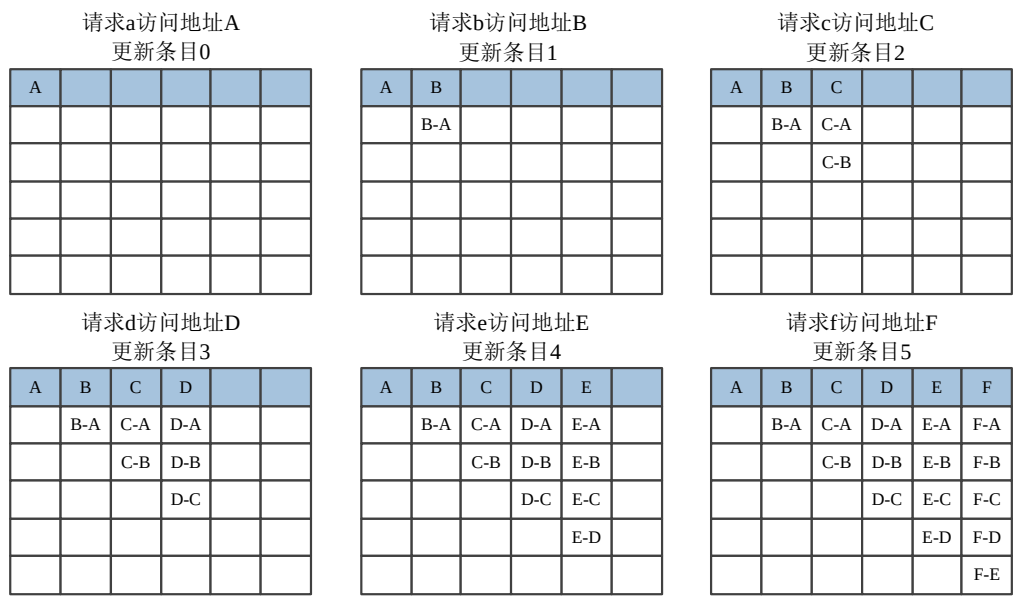


图 4.7 跨步数据流检测实例

4.5 本章小结

本章节详细阐述了本文预取方案的设计思想与实现细节，首先给出了国产处理器 64 位访存虚拟地址组成结构，通过对不同地址字段在处理器访问 Cache 时所代表的实际意义的分析，具体确定了预取部件所关注的 Cache 行地址。随后介绍了国产处理器的访存路径，选择将预取部件添加到内核中的数据 Cache 管理部件，紧接着给出了预取部件的组成、结构以及完成一次预取操作所需要的大致流程，详细叙述了各个预取子模块的功能，并对各子模块相应的端口信息进行了必要的说明。其中，顺序数据流与跨步数据流被分开检测，同时数据流识别结果统一记录在数据流预取条目中，这既避免了不同步长数据流检测过程可能存在的干扰，也便于后续预取控制模块对数据流的管理与更新。预取请求缓冲则负责暂存生成的预取请求，防止预取请求不经仲裁直接上流水线影响正常指令的执行。最后本章还给出了几种数据流检测的实例，更深一步直观揭示了预取部件的工作原理。

第五章 性能评测与数据分析

在国产某型号具有自主指令集架构的处理器核心中完成预取部件的硬件设计后，本文接下来要做的主要工作就是对设计出的预取部件进行性能评估。以下本章将对测试过程中的测试环境、测试与评估方案作出介绍，并对最终的实验测试结果数据进行分析，总结出预取部件对处理器性能实际的提升情况。

5.1 测试环境

现代计算机系统具有规模巨大、软硬件繁多以及设计复杂等特性，因此对处理器的性能测试是繁琐且富有挑战性的。一般而言，评估预取部件对处理器性能提升的度量，业界通常采用专业模拟器或搭配预取部件的处理器在仿真测试平台运行标准测试基准程序得出。本文选择后者，以下将对仿真测试所使用的基准程序、处理器模型以及硬件加速器做出必要的介绍。

5.1.1 性能测试基准程序(Benchmark)

工业界、学术界和研究院对处理器性能评估方法的共通需求直接促进了标准性能测试基准程序的发展。测试基准程序是一个使用高级程序语言编写的程序集合，它试图针对不同领域的计算机系统性能提供一种可靠且具有代表性的测试。目前用于 CPU 执行效率评估的程序主要如下，包括：SPEC CPU 系列、SPEC Cjbb 系列、Sysbench 以及 PCmark 等。

本文采用目前使用最为广泛的 SPEC CPU2006 测试基准程序作为实验的仿真激励。标准性能评估公司 SPEC(Standard Performance Evaluation Corporation)是一家在 1988 年由许多计算机销售商共同成立于美国的非营利性合作组织，旨在“生产、建立、维护和认可一套标准化的计算机性能基准”。1989 年，该组织推出第一版重点面向 CPU 性能评估的软件程序，并将其命名为 SPEC 89。经过多年的发展，SPEC 于 2006 年推出了全新的处理器全面评估软件 SPEC CPU2006。新的程序与之前 SPEC CPU2000 相比，代码量超过了 300 万行且涵盖的科学计算范围明显更加丰富。SPEC CPU2006 共包括 12 个整数软件代码(CINT 2006)和 17 个浮点软件代码(CFP2006)，分别对处理器执行整数与浮点数的性能作出测评。

SPEC CPU2006 衡量处理器性能的方法有两种：1、CPU 完成单一任务的速度；2、对程序执行中的吞吐量、容量以及速率的测量。SPEC CPU2006 涵盖了以计算科学为主的处理器应用领域，通过一系列实际应用的程序，确保所得出的测试数据真实可靠的反映出处理器的性能水平。

目前 SPEC CPU2006 基准测试程序被广泛运用于国内外超算、服务器以及桌面系统的性能测试领域。下表 5.1、5.2 分别对 CINT2006、CFP2006 各个测试子程序的数字编号、源代码语言以及各个子程序所执行的任务等基本信息进行了归纳。

表 5.1 SPEC CPU2006 整数测试程序

子程序	程序语言	组件/原型	描述
400.perlbench	ANSI C	Perl 语言	三个 script 负载
401.bzip2	ANSI C	压缩	负载包括六个部分
403.gcc	C	C 语言编译器	9 组 C 代码进行编译
429.mcf	ANSI C	组合优化	车辆调度的程序
445.gobmk	C	人工智能：围棋	围棋
456.hmmmer	C	基因序列搜索	使用基因识别方法进行基因序列搜索
458.sjeng	ANSI C	人工智能：国际象棋	国际象棋
462.libquantum	ISO/IEC	物理学：量子计算	量子计算的库文件
464.h264ref	C	视频压缩	使用两种配置对源文件进行编码
471.omnetpp	C++	离散事件仿真	模拟了一种大型 CSMA/CD 以太网协议
473.astar	C++	寻路算法	完成三种版本的 2D 寻路算法
483.xalaxcbmkXML 处理	C++	XML 处理	将 XSL 表与 XML 文档转换为 HTML 文档

表 5.2 SPEC CPU2006 浮点测试程序

子程序	程序语言	组件/原型	简要说明
410.bwaves	Fortran 77	流体力学	模拟计算三维跨音速粘性流冲击波
416.gamess	Fortran	化学领域	等效平均场计算
433.milc	C	力学领域	量子色动力学研究
434.zeusmp	Fortran77/REAL	物理学：计算流体力学	模拟计算统一磁场中 3D 冲击波
435.gromacs	C & Fortran	分子力学/生物化学	分子力学计算套件
436.cactusADM	Fortran 90, ANSI, C	物理学：广义相对论	求解爱因斯坦方程
437.leslie3d	Fortran90	流体力学	计算湍流的流体力学组件
444.namd	C++	分子/生物	大型的生物分子并行计算程序
447.dealII	C++	分析学	对自适应有限元进行分析的 C++库
450.soplex	ACSI C++	线性编程、优化	求解方程使用单纯形的方法
453.povray	ISO C++	影像光线追踪	光线追踪渲染的一个程序
454.calculix	Fortran 90 & C	结构力学	用于非线性与线性的三维结构力学的分析软件
459.GemsFDTD	Fortran 90	物理电磁学	使用 FDTD 方法对 Maxwell's equations 进行求解
465.tonto	Fortran 95	化学领域	研究化学问题的程序包
470.lbm	ANSI C	流体动力学	使用 LBM 模拟非压缩流体
481.wrf	Fortran 90 & C	气象分析	使用定制模型，对 NCAR 数据进行计算
482.sphinx3	C	语言识别	智能检测程序

5.1.2 处理器模型

本文采用国产某型号自主指令集架构高性能处理器作为实验中搭载预取部件的处

理器模型。该芯片为国产针对服务器应用领域需求所设计的高性能处理器。其主要性能特性如下：

- 1、并行资源丰富，设计有多种线程工作方式；
- 2、超标量双流水线设计，乱序发射、乱序执行等方式；
- 3、片上集成两级缓存，一级 Cache 数据与指令独立设计，二级 Cache 数据指令共享，片外共享三级 Cache。硬件支持 Cache 数据一致性。

下表 5.3 给出了本文所使用的国产处理器模型相关参数信息以及预取过程中一些相应的指标参数信息。

表 5.3 处理器参数及预取相应指标信息

性能参数	配置信息
一级指令 Cache	8 路组相连，容量 64KB，偶校验，LRU 淘汰算法
一级数据 Cache	8 路组相连，容量 64KB，ECC 校验，LRU 淘汰算法
二级 Cache	8 路组相连，容量 512KB，ECC 校验，LRU 淘汰算法
三级 Cache	48MB，分为 16 个分体，每个分体 3MB 采用 12 路组相联，ECC 校验，LRU 淘汰算法
Cache 行大小	128 个字节(byte)
预取地址	虚拟地址
预取数据大小	Cache 行
预取请求优先级	低于正常访存请求
访存命中延时	一级数据 Cache 访问延时不大于 4 拍，二级 Cache 访问延时不大于 13 拍

5.1.3 硬件仿真加速器

传统上意义上来说，对于处理器的性能测评主要使用三种方法：软件模拟验证、硬件仿真加速器与 FPGA 平台验证。软件模拟验证主要工作原理是在工作站的处理器模型上对信号和触发器中的数据进行布尔运算，但随着半导体制造工艺的提升，芯片上晶体

管集成度规模变大，软件方法的局限性也凸显出来，其在仿真速度方面下滑巨大，严重影响了仿真的效率且结果数据的精度也不太理想。FPGA 平台验证相比于另外两种测评方法速度上有着巨大的优势，但该方法具有开发周期综合耗时长、不受控、以及错误定位不准确等问题。硬件仿真加速方法在速度上介于另外两者之间，但具有良好的可观性与可控性，并且能够准确的定位错误位置。基于运行速度与仿真精度的综合考量，本文最终选择采用硬件仿真加速器来对处理器性能进行验证测试。

本实验具体选用 Cadence 公司生产的硬仿加速器 Palladium XP 作为评估 CPU 运行测试程序性能的验证平台。Palladium XP 仿真加速器具有编译时间短、运行速度快以及定位错误信息准的优势。该设备中具体参数与工作特性如下：

- 1) 最大容量达 2 Billion Gates。
- 2) 运行速度最快达 4MHz。
- 3) 灵活的资源共享，最多支持 512 用户。
- 4) 综合编译速度 30 Million Gates/小时。
- 5) 最大 Memory 容量 2TB。
- 6) 最多外部 IO 接口数量 147K 根。
- 7) 支持基于断言的功能覆盖率分析。
- 8) 支持功耗分析。

仿真加速器的工作原理为：在芯片仿真验证时将待运行的数字电路进行计算以验证其结果是否满足预期要求，其中 Verilog HDL 所要实现的功能，在仿真时直接编译成能够操作的布尔表达式，于是芯片所要实现的功能就可以等价于加速器中实际执行的布尔表达式结果。加速器平台的系统组成结构如下图 5.1 所示，Palladium XP 对 RTL 代码进行前期编译，而前端工作站则负责 Testbench 进行综合、运行调试环境以及对生成的测试数据结果进行处理。前端工作站与 Palladium XP 之间的通信主要由网络控制卡负责，它们之间的连接方式有三种：

- 1) 以太网交换机：负责启动网络控制卡，设备初始以及发送警报信息。
- 2) 光纤交换机：则用于前端工作站和 Palladium XP 之间的数据传输(逻辑下载与波形上传等)。
- 3) SA(Simulation Acceleration)卡：用于模拟加速环境下的数据传输。

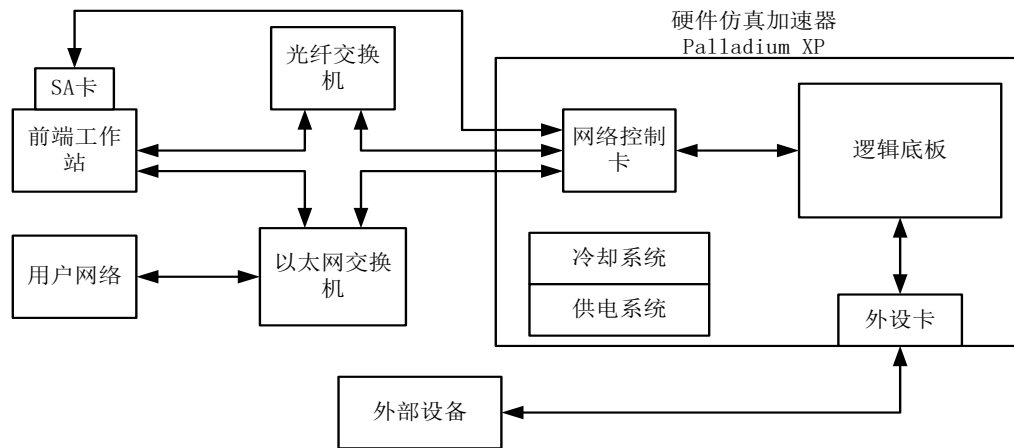


图 5.1 硬件仿真加速器系统结构

硬件仿真加速器的使用流程分为两步：第一，将待验证的逻辑设计(本文中为加入预取部件前后的处理器模型)经过逻辑综合、编译、生成目标文件；第二，使用相应仿真环境运行仿真器。最后，当测试程序运行完毕后，统计并分析相关数据信息并进行分析，从而最终确定加入预取部件后处理器性能实际变化情况。

5.2 测试与评估方案

本文以国产某型号自主指令集架构高性能处理器中，使用 Verilog HDL 硬件描述语言完成硬件预取部件的 RTL 代码设计，并实例化到处理器核心的数据 Cache 管理部件，最终形成本实验所需的含有硬件预取部件的处理器。给测试平台硬仿加速器配置相应的验证环境，对比加入硬件预取部件前后处理器运行测试程序的性能差异。考虑到时间因素与论文进度，本文仅选择了 SPEC CPU2006 中的 7 个整数程序与 3 个浮点程序作为实际仿真过程中的测试激励。性能测试的流程见图 5.2 所示。

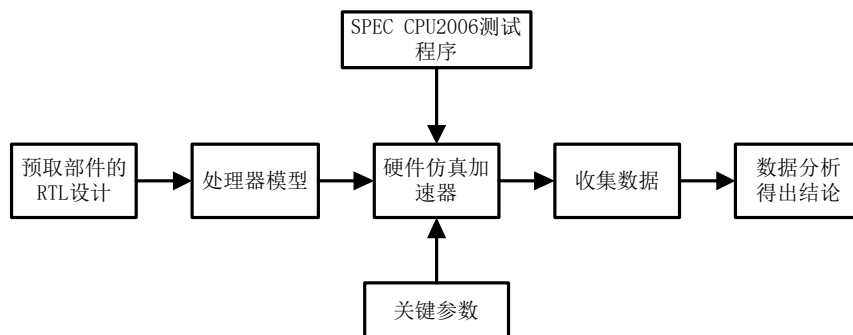


图 5.2 性能测试流程

仿真测试结束后，本文数据中选择 IPC、各级 Cache 的命中率的变化情况作为处理器性能评估的参照标准。IPC 英文全称“Instruction Per Clock”，中文直译为每个时钟周期

指令数，即每个时钟周期处理器所能执行完的指令数量。目前常用的处理器性能标准评估公式为：处理器性能=IPC * 频率。因此，在频率一定的情况下，IPC 值的高低就是评价处理器性能高低的关键参数。随着预取部件的加入，处理器执行指令操作时能够更快的获得所需的数据，因此每个时钟周期的处理的指令数量在理论上必然增加，于是 IPC 值变化的多少，最能够直观的表现出预取部件对于处理器性能的影响。Cache 的命中率 (Cache Hit Rate)则是另一个衡量处理器访存效率的关键指标参数。预取策略的本质是对将来即将访问数据的地址进行预测并提前几个时钟周期将该数据从低层次的存储设备取至离处理器更近的 Cache 中，从而确保处理器在 Cache 中获得所需数据的概率增大，避免了访存过程中出现的处理器停滞，最终达到隐藏访存延时的目的。因此，Cache 命中率是否提升也能够表明本文设计的预取是否有效。

5.3 测试结果与分析

本文实验中，以添加预取部件后的处理器为载体，使用硬件仿真加速器，在预先配置的验证环境下运行 SPEC CPU2006 测试基准程序，并以相同环境下运行没有添加预取部件的处理器运行相同程序作为参照，统计仿真结果，并依据数据展开分析与处理，从而得出结论并发现其中可能存在的不足。

图 5.3 显示了添加预取部件的处理器模型与没有添加预取部件的处理器模型在相同配置环境下测试得出的 IPC 性能数据结果。从图中可以看出，由于预取部件的加入，处理器运行 7 个整数程序与 3 个浮点程序后，IPC 数值都有着不同程度的提升，这表明本文设计的硬件预取部件对处理器性能的提升是有着积极意义的。

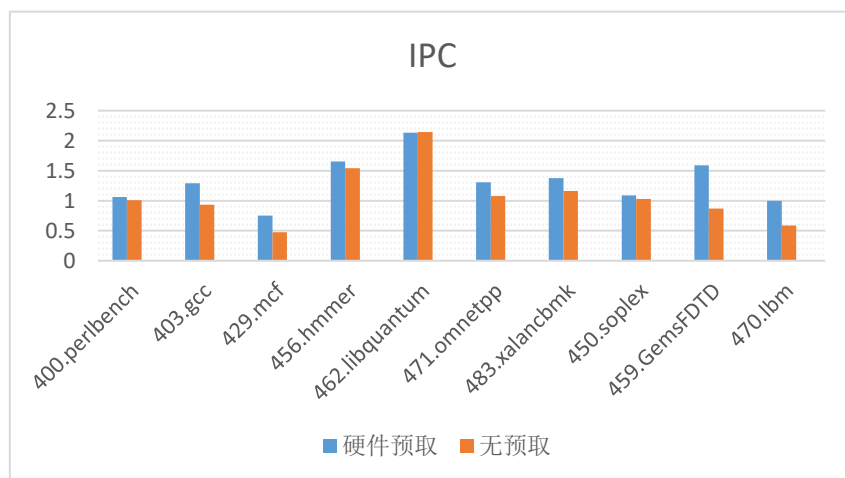


图 5.3 处理器 IPC 性能对比

通过图 5.4 中给出的 IPC 性能的提升百分率，更能够直观的说明预取部件对处理器 IPC 性能的影响。10 个测试程序中，除了 462.libquantum 之外，其余运行时 IPC 都实现了一定程度的正增长。这证明了本文基于程序中数据流访存模式的预取部件，能够完成数据流识别的预期功能并预先获取有用数据，加速了处理器每个时钟周期对指令的处理速度，实现系统整体性能的提升。其中，运行浮点测试程序 459.GemsFDTD 与 470.lbm 的 IPC 提升率更是达到了 70%以上，这表明此类浮点程序具有良好的局部性结构，程序运行时出现类似数组等形式的数据流访存模式的概率较大。对于 IPC 没有提升的程序 462.libquantum 来说，可能的原因有两个：1、其本身运行时 IPC 已经很高，处理器执行指令速度已经达到饱和；2、程序本身局部性不强，可能存在大量的随机访问模式，本文的预取策略无法有效的对其进行识别与访问预测。再通过从图 5.3 可以分析出，由于没有加入预取部件时程序 462.libquantum 的 IPC 已经超过了 2，因此其性能无法提升的原因很大概率上属于前者。

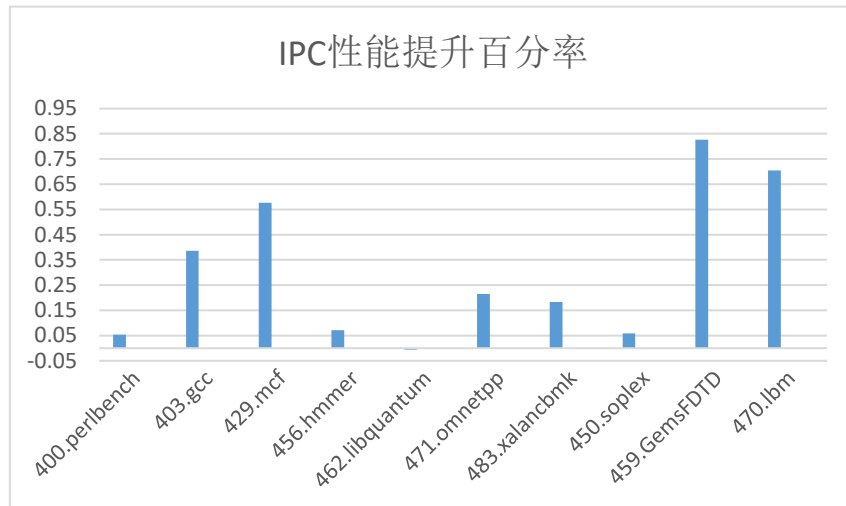


图 5.4 配置预取后处理器 IPC 性能提升百分率

本实验处理器内核设置了大小相等的分离式一级数据 Cache(DCache)与一级指令 Cache，由于本文预取部件针对的是数据预取，因此下图 5.5 仅仅统计了加入预取部件后一级数据 Cache 与没有预取部件时一级 DCache 访存命中率的比较(本文以下所指命中率均为平均命中率)。一级数据 Cache 离处理器最近、容量最小且延时最低，其中存放的数据信息大多为处理器刚刚使用过或与正在被访问数据地址相近的数据。从图中直观的看出，无预取条件下，处理器运行不同测试程序时，Cache 的命中率已经很高，这说明大部分程序局部性良好，处理器访存过程中所需处理的信息大概率都能在 Cache 中命中，

因此预取部件加入后，Cache 命中率提升有限。但对于原本 Cache 命中率相对较低的程序 429.mcf 与 470.lbm 来说，预取部件仍然能带来一级 DCache 命中率较为显著的提升。

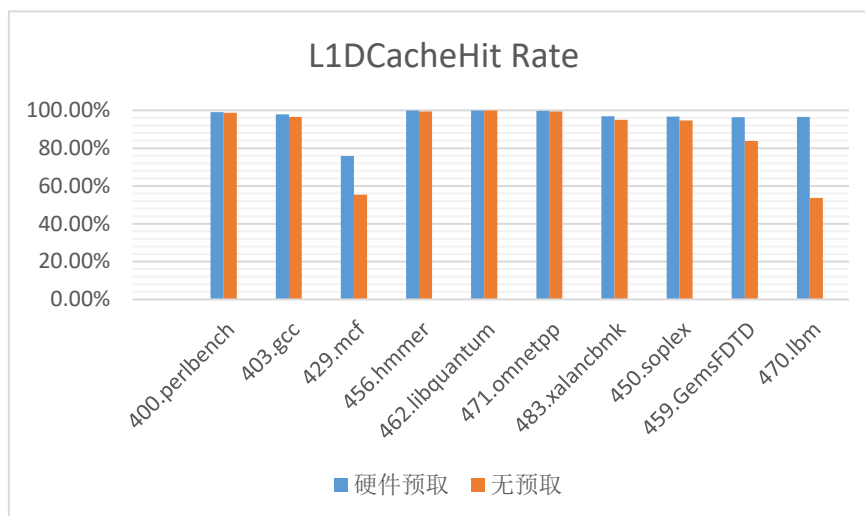


图 5.5 处理器运行不同程序 L1DCache 命中率对比

下表 5.4 更加清楚的给出了各个 SPEC CPU2006 程序课题访存时，一级 DCache 的命中率，并计算出了预取部件带来的实际命中率提升百分比。

表 5.4 各个程序一级 DCache 命中率具体信息

task_name	DCache 命中率		
	无预取	硬件预取	性能提升百分比
400.perlbench	98.74%	99.07%	0.3%
403.gcc	96.5%	97.82%	1.3%
429.mcf	55.41%	75.89%	36.9%
456.hmmer	99.41%	99.90%	0.5%
462.libquantum	99.84%	99.93%	0.1%
471.omnetpp	99.33%	99.78%	0.5%
483.xalancbmk	94.91%	96.89%	2.1%
450.soplex	94.64%	96.66%	2.1%
459.GemsFDTD	83.77%	96.30%	15.1%
470.lbm	53.73%	96.57%	79.8%

对于访存过程中存储系统一级 DCache 命中率提升最大的程序 470.lbm 来说，下图

5.6、5.7 进一步分别给出了不加预取与增加预取部件后，整个测试过程中处理器访问一级 DCache 命中率的变化情况。两幅图中，纵坐标表示为一级 DCache 命中率大小，横坐标则表示为访存过程中选取统计的时钟周期间隔，其中每一个单位刻度代表五万个时钟周期(本文每隔五万拍时钟周期统计一次指标参数)。从图 5.6 中可以看到，在横坐标统计单位 13777 到 22961 区间内(实际时钟周期为 $13777*5w$ 拍到 $22961*5w$ 拍之间)，系统中一级 DCache 的访存命中率比较低均在 40%以下，随后命中率虽然有所上升，但大多保持在 60%的上下浮动，整个测试过程中其平均命中率也仅有 53.73%，因此预取部件对该程序运行时的 DCache 访存命中率有潜在提升的能力。

再通过图 5.7 中的数据可以看出，随着硬件预取部件的加入，整个访存过程中基准测试程序 470.lbm 的一级 DCache 的平均命中率已经达到 96.57%，命中率提升百分比为 79.8%。图 5.6 中程序 DCache 命中率较低的执行区间，在本文预取策略的作用下得到了明显的改善。预取部件对程序 470.lbm 运行时，一级 DCache 命中的提升的原因分析如下：预取部件对程序 Cache 命中率的提升与其本身访存过程中的访存模式密切相关，程序 470.lbm 在执行过程中，相当一部分指令命中的数据都不在 Cache 当中，处理器需要等待下一级 Cache 中的数据传入当前 Cache 才能继续完成相应操作，因此造成了 DCache 的命中率整体偏低。从数据结果与预取策略结合来看，当预取部件加入后，DCache 的命中率得到了显著的提升，因此我们可以确定程序 470.lbm 在横坐标 13777 后运行过程中内存访问有着大量的数据流访存模式(顺序流/跨步流)，预取部件识别数据流并发出预取请求取回数据，这也再次印证了本文预取策略能够有效的降低处理器的访存延时。

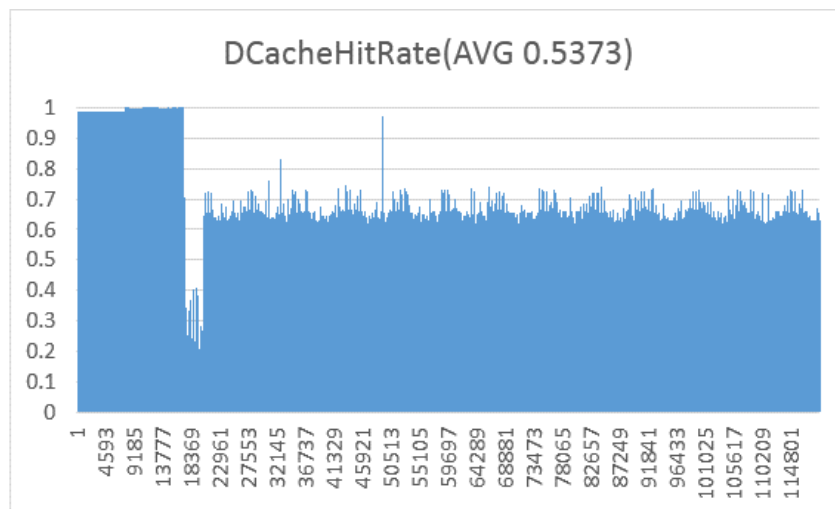


图 5.6 无预取时程序 470.lbm 访存过程中 DCache 命中率情况

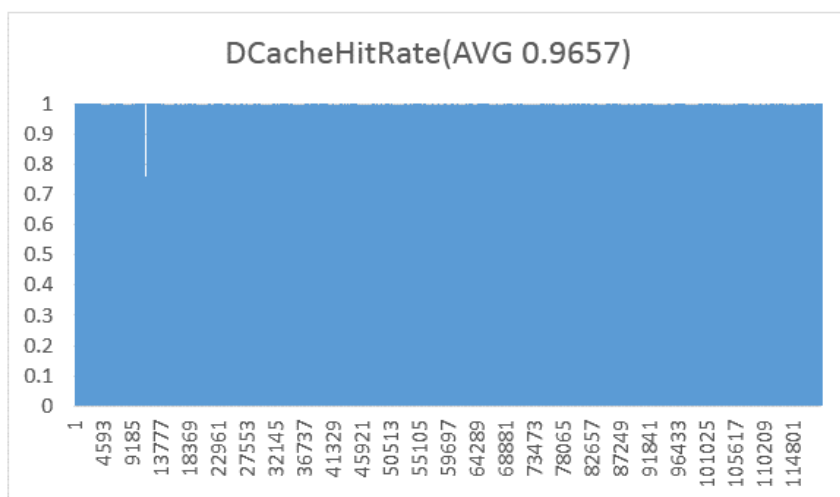


图 5.7 增加预取部件后程序 470.lbm 访存过程中 DCache 命中率情况

图 5.8 给出了程序执行时,存储系统二级 Cache 访存命中率的对比图。与一级 DCache 相比,二级 Cache 的存储容量与访存延时变大,再加上每个测试程序本身访存行为特征的不同,因此各个程序无预取时二级 Cache 命中率相差较大。在加入预取部件后,对于无预取时本身二级 Cache 命中率就较高的程序,加入预取部件后命中率提升已不明显,如程序 456.hmmmer。对于无预取时二级 Cache 命中率就处于较低水平的程序来说,由于程序本身局部性较差,程序访存过程中预取部件无法识别出触发预取所需的数据流访存模式,因此其命中率仍然无法得到有效提升,此类情况包括 400.perlbench、462.libquantum、471.omnetpp 以及 459.GemsFDTD 等测试程序。其中,测试程序 471.omnetpp 在预取部件加入后二级 Cache 命中率甚至还出现了一定程度的下降,其原因可能是预取部件发出无效的预取请求占据了宝贵的总线带宽,阻塞了正常访存指令的执行,从而使得正常访问的请求数据返回的延时加长,最终造成了系统一定程度上性能的损失。

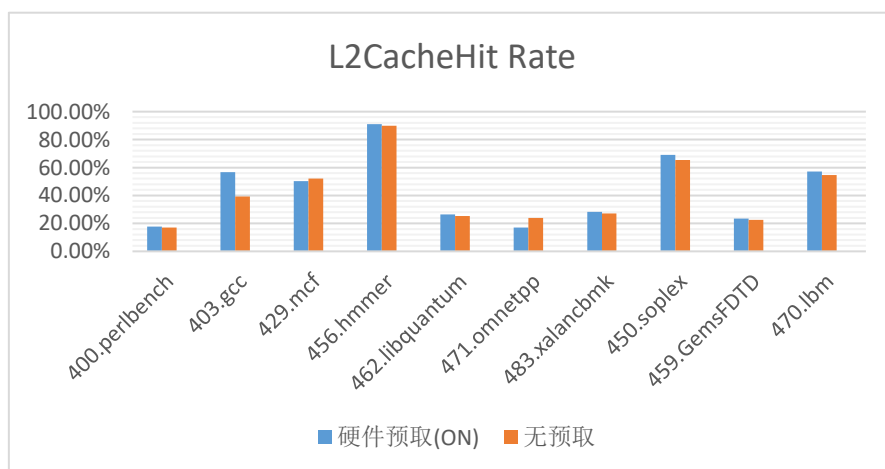


图 5.8 处理器运行不同程序 L1DCache 命中率对比

总体来看, 预取部件对于各程序的二级 Cache 命中率还是有着积极作用的, 程序 470.lbm、450.soplex 以及 403.gcc 等都有着一定程度的提升。由下表 5.5 看出, 其中程序 403.gcc 命中率提升百分比达到了 44.1%。表 5.5 具体统计了各个程序详细的二级 Cache 命中率情况。

表 5.5 各个程序 L2Cache 命中率具体信息

task_name	二级 Cache 命中率		
	无预取	硬件预取	性能提升百分比
400.perlbench	16.96%	17.68%	4.2%
403.gcc	39.36%	56.74%	44.1%
429.mcf	52.11%	50.30%	-3.4%
456.hmmer	89.91%	91.07%	1.3%
462.libquantum	25.35%	26.47%	4.4%
471.omnetpp	23.98%	16.95%	-29.3%
483.xalancbmk	27.10%	28.25%	4.2%
450.soplex	65.29%	68.99%	5.7%
459.GemsFDTD	22.64%	23.46%	3.6%
470.lbm	54.73%	57.18%	4.5%

本文预取部件在对一级数据 Cache 与二级 Cache 实现预取之外, 还设置了预取距离更大的三级 Cache 预取, 但是由于三级 Cache 本身容量过大, 而所选的测试基准程序与实际应用的程序相比仍然过小, 无法真实的衡量大容量三级 Cache 在程序运行时的存取行为。因此本文中三级 Cache 的预取仅起到为高层次 Cache 供数的作用, 而不统计其本身的命中率。

5.4 本章小结

本章主要通过实验过程中的测试环境、测试方案以及最后的测试数据三个小节来对添加预取部件后的处理器模型进行系统的性能测评。从结果数据来看, 本文所提出的基于应用程序数据流访存模式的预取策略能够加速处理器每个周期执行指令的速度, 提升

了程序运行的效率。另外，在 Cache 命中率方面，对于容量较小的一级数据 Cache，加入本文预取部件后，各个测试程序运行时的缓存命中率都得到了很好的提升，其中程序 470.lbm 的提升率更是达到了 79.8%。对于二级 Cache 来说，部分测试程序命中率提升不太明显，其中程序 471.omnetpp 由于本身局部性较差或可能的无效预取请求挤占总线带宽等因素下，出现了造成一定程度的性能损失。但通过测试结果整体来看，大部分测试程序尤其是涉及科学计算与工程领域，数据流访存模式在访存过程中是普遍存在的。因此，本文预取方案能够有效降低数据访问的延时，为处理器性能带来可观的提升。

总结与展望

现代计算机领域中,由于处理器与主存储器之间访存速度巨大的发展差异所带来的“存储墙”问题,严重制约了计算机系统的整体运行速度。随着多核多线程处理器的蓬勃发展,对隐藏访存延时也提出了更高的要求,大量研究也已经表明,多核多线程处理器平台中“存储墙”的问题也变得的更加严峻。随着计算机架构中层次型存储结构的引入,“存储墙”问题得到了一定程度的缓解。但由于受到价格、功耗以及硬件开销等因素的限制,存储结构中多级 Cache 的容量大小与主存相比仍然是微不足道。因此,处理器访存过程中一旦出现 Cache Miss 就不得不从主存储器中取回所需处理的数据,而这一过程消耗的时钟周期往往高达几百拍。于是,致力于提高 Cache 命中率的方法就成了隐藏处理器访存延时的新的研究方向,预取技术也正是在这一背景下迅速发展至今。预取技术现今主要有三种实现策略,且各个方法互有优缺点。从目前来看,软件预取与硬件预取应用最为广泛,国内外大部分主流商用处理器都有着相应的软件预取指令,并在处理器核心中设计了硬件预取部件。考虑到预取准确性与其效率,本文选择从硬件预取展开研究。

本文所做的主要贡献如下:

1) 提出一种在处理器工作时检测并识别程序中数据流访存模式的数据硬件预取策略,制定了对硬件预取部件进行评估的方案。

2) 对内存中程序访存行为特征进行了分析,例如处理器访问内存程序信息的地址翻译方式、程序的局部性原理以及程序中存在的几种访存模式。同时,对现有访存模式与预取策略之间的适配性与设计复杂度进行了分析,最后,结合本文所使用的国产处理器的特性与程序代码实例讨论的结果,最终确定了基于数据流访存模式设计预取部件的思路。

3) 在国产某型号自主指令集高性能处理器中,完成本文预取方案的设计并例化到内核中的数据 Cache 管理部件。本文对顺序数据流与跨步数据流进行了单独识别,避免了数据流之间识别干扰而造成的预取部件准确率下降。同时,预取部件设置了多个预取条目,能够实现多个数据流的并行检测,提升了预取部件的效率。最后,本文还设计有单独存放预取请求的预取缓冲,并规定了预取请求上仲裁流水线的优先级,最大限度保证了正常访存指令不会受到预取请求的干扰。

4) 使用标准处理器性能评估软件 SPEC CPU2006 作为测试程序, 选择硬件仿真加速器 PalladiumXP 作为实验仿真平台, 确保了测试数据的可靠性与直观性。在预取性能评估方面, 本文选取 IPC 与 Cache 命中率这两个衡量处理器性能的重要指标参数, 通过具体的测试数据对设计性能进行了全面的分析与讨论。结果表明, 本文提出的预取方案能够有效的提升处理器执行指令的效率, 同时也提高了各级缓存的命中率, 很好的实现了隐藏系统访存延时的预期目标。

在有着良好局部性的前提下, 例如科学计算与工程应用等数据流访存模式占比较大的程序中。本文所提出硬件预取方案有着很好的适用性, 同时也有着不错的准确性与高效性。但本文仍然有着不足之处, 其后续研究工作可以从以下几个方面继续展开探索。

1) 应用程序访存模式自适应: 本文预取策略主要依赖于科学计算领域中所存在的大量数据流访存模式, 其适用性难免存在一定的限度。在数据流程序模式之外, 程序运行还存在着丰富的多样性, 无法保证指令执行期间程序访存模式保持良好的一致性。但目前包括本文在内大多数硬件预取机制都有着很强的针对性, 在某些访存模式下对处理器性能有着不错的提升, 但却难以推广到其它的访存模式。因此, 对程序访存模式自适应的硬件预取策略的研究就显得十分必要, 使得预取策略根据程序运行过程中访存模式的变化, 动态的调整预取方案, 从而达到预取策略自适应访存模式的目的。

2) 针对多级 Cache 设计不同预取策略: 现代计算机层次型存储结构一般包括多级 Cache 结构, 各级 Cache 之间由于容量、访存延时的不同, 导致结构差异巨大。本文中多级 Cache 采用相同的预取策略, 各级 Cache 之间预取请求之间仅有预取距离的差异。从数据结果来看, 本文预取部件对一级数据 Cache 命中率的提升率要优于二级 Cache, 其中二级 Cache 的预取中部分程序还出现了性能损失的情况。因此, 在接下来的研究工作中, 针对二级 Cache 以及三级 Cache 的工作原理与数据的一致性, 单独设计满足二级/三级 Cache 的预取策略也是具有积极意义的。

3) 多核多线程处理器领域: 随着芯片设计水平与制造工艺的创新, 处理器中并行的内核与线程数愈发丰富, 这为预取技术的研究提供了新的方向。多核与多线程体系结构下, 各个内核独享缓存具有复杂的数据一致性、多个线程之间数据预测难以保证同步以及如何平衡不同逻辑单元的资源共享等问题, 都会对整个预取策略的效率与准确性带来挑战, 这也是研究人员将来亟待处理的问题。

参考文献

- [1] Binny S.G, Dharmendra S.M. Sequential Prefetching in Adaptive Replacement Cache[C]. Proc 2005 Annual Technical Conference. 2005: 293-308
- [2] 贺云波. 基于 Boom 的硬件预取技术的研究与实现[D]. 陕西西安: 西安电子科技大学, 2019
- [3] 李文. 存储控制系统性能优化技术研究[D]. 北京: 中国科学院计算技术研究所, 2005
- [4] 贾迅, 翁志强, 胡向东. 基于流访问特征的多级硬件预取[J]. 计算机工程, 2016(1): 51-55
- [5] 裘雪红, 盛立杰, 张剑贤. 计算机组成与系统结构[M]. 陕西西安: 西安电子科技大学出版社, 2020: 20-60
- [6] 马汝亮. 高性能处理器存取关键技术的设计与优化[D]. 上海: 上海交通大学, 2013
- [7] Bryant R.E, Hallaron D.R. 深入理解计算机系统[M]. 原书第三版. 龚奔利, 贺莲. 北京: 机械工业出版社, 2020: 421-450
- [8] Jew E. The IBM Power8 Processor Core Microarchitecture[G]. IBM DeveloperWorks AIX Virtual Group, 18th 2016
- [9] Jimenez V, Cazorla F. Adaptive Prefetching on Power7: Improving Performance and Power Consumption[J]. ACM Trans Parallel Comput, 2014,4(1): 25-50
- [10] Megan G. IBM System Blue Gene Solution Blue Gene/Q Application Development[G]. June 2013
- [11] Jack D. Inside Intel Core Microarchitecture and Smart Memory Access[G]. Technology @ Intel Magazine, 2016
- [12] Intel. Intel 64 and IA-32 Architectures Optimization Reference Manual[G]. June 2016
- [13] Young K, Shekita E. An intelligent I-Cache prefetch mechanism[J]. IEEE International Conference on Computer Design, 1993, 23(1): 44-49
- [14] 赵祥. 基于应用程序访存模式的硬件自适应预取技术的研究[D]. 湖南长沙: 国防科技大学, 2014
- [15] Porterfield K. Software Methods for Improvement of Cache Performance on

- Supercomputer Applications[D]. Houston: Rice University, 1989
- [16]Mittal S. A Survey of Recent Prefetching Techniques for Processor[D]. Mumbai: Indian Institute of Technology, 2016
- [17]Friedrich, Sweet M. Design and implementation of the Power5 microprocessor[C]. In Proceedings of the 41st Annual Conference on Design Automation, 2004: 670-672
- [18]David C, Ken K, Allan P. Software Prefetching[C]. Proceedings of the 4th ASPLOS Conference, 1991: 40-52
- [19]Alan S. Cache Memories[J]. ACM Computing Surveys Volume, 1989(9): 85-94
- [20]贾迅, 胡向东, 尹飞. 申威处理器硬件数据预取技术的实现[J]. 计算机工程, 2015, 37(11): 2013-2017
- [21]Chen T.F, Baer. Effective Hardware-Based Data Prefetching for High-Performance Processors[J]. IEEE Transactions on computers, 1993
- [22]Tyson G, Austin T.M. Improving the accuracy and performance of memory communication through renaming[C]. In 30th Annual International Symposium on Microarchitecture, Dec, 1997: 218-227
- [23]张荣芸. 浅析缓存预取技术[J]. 现代计算机. 2011(06): 39-40
- [24]Dahlgren, Dubois F.M, Stenstrom P. Fixed and Adaptive Sequential in Shared-memory Multiprocessors[C]. Proc International Conference on Parallel Processing, 1993: 56-63
- [25]Jouppi N.P. Improving directed-mapped cache performance by addition of small fully-associative cache and prefetching buffers[J]. ACM SIGARCH Computer Architecture News, 1990, 18(3): 363-373
- [26]Palacharla S, Kessler R.E. Evaluating stream buffers as a secondary cache replacement[C]. Proceedings of the 21st annual international symposium on Computer Architecture, 1994: 24-33
- [27]Kim T, Zhao D.L, Veidenbaum A. Multiple stream tracker: a new hardware stride prefetcher[C]. In Computing Frontiers, 2014: 34-40
- [28]Zhang C, McKee S.A. Hardware-Only Stream Prefetching and Dynamic Access Ordering[C]. In Proc of the 14th Annual International Conference on Supercomputing,

- 2000: 54-60
- [29] Michaud P. A Best-Offset Prefetcher[C]. 2nd Data Prefetching Championship, Jun 2015: 36-42
- [30] Pugsley S.H, Chishti Z, Wilkerson C, et al. Sandbox prefetching: safe run-time evaluation of aggressive prefetchers[C]. In HPCA, 2014
- [31] Michaud P. Best-Offset Hardware Prefetching[C]. International Symposium on High-Performance Computer Architecture, Mar 2016
- [32] 隋然, 张铮, 张为华. 预取技术分析[J]. 电子应用技术, 2015, 41(6): 9-12
- [33] Joseph D, Grunwald D. Prefetching using Markov predictors[C]. In Proceedings of the 24th Annual International Symposium on Computer Architecture, 1997: 252-263
- [34] Wenisch T.F, Somogyi S, Hardavellas K, et al. Temporal Streaming of Shared Memory. In Proc of the 32nd Annual International Symposium on Computer Architecture, June 2005
- [35] 方娟, 张洪波. 多核处理器预取策略的研究[J]. 微电子学与计算机, 2009, 27(8): 75-77
- [36] Chappell R, Stark J, Kim S, et al. Simultaneous subordinate microthreading (ssmt)[C]. In Proceedings of the International Symposium on Computer Architecture, May 1999
- [37] Jain K. Linearizing irregular memory accesses for improved correlated prefetching[C]. Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture, 2013: 247-259
- [38] 张百达. 一种软硬结合的预取技术的研究[D]. 湖南长沙: 国防科技大学, 2007
- [39] Zhang Z, Torrellas J. Speeding up Irregular Applications in Shared-Memory Multiprocessors: Memory Binding and Group Prefetching[C]. Proc 22th International Symposium on Computer Architecture, Santa Margherita Ligure, Italy 1995: 188-199
- [40] Vander S.P, Hsu W.C, Lilja D J. When Cache are not Enough: Data Prefetching Techniques[J]. IEEE Computer, July 1997, 30(7): 23-27
- [41] Wang Z.L, Burger D. Guided Region Prefetching: A Cooperative Hardware/Software Approach[C]. Proceedings of the 30th Annual International Symposium on Computer Architecture, 2003

- [42]唐言. 一种软硬件结合的预取技术研究[J]. 中国新技术新产品, 2010(10): 31-32
- [43]Ayers G, Litz H, Kozyrakis C. Classifying Memory Access Patterns for Prefetching[C]. In Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, March 2020: 16-20
- [44]Badawy A.H, Yeung D, Tseng C.W. The Efficacy of Software Prefetching and Locality Optimizations on Future Memory Systems[C]. Journal of Instruction-Level Parallelism, 2004: 10-36
- [45]Mowry T.C, Lam M.S, Gupta A. Design and Evaluation of a Compiler Algorithm for Prefetching[J]. ACM SIGPLAN Notices, 1992, 27(9): 62-73
- [46]张乾龙, 侯锐, 杨思博. 链表结构反馈预取[J]. 高技术通讯, 2019, 29(1): 27-36
- [47]肖俊华, 冯子军, 章隆兵. 片上多处理器中基于步长和指针的预取[J]. 计算机工程, 2009, 35(40): 58-60

致谢

光阴飞逝，转眼之间三年的研究生生涯即将徐徐落下帷幕。在学校的三年研究生学习阶段中，我不仅学习到了专业知识技能，而且也在做人的道理上上获益良多。求学期间，我结识了很多优秀的同学与老师，他们都在学习与生活当中给了我很大的帮助，让我看到了人生中充实而精彩的一面。在本文即将完成之际，真诚的向所有帮助过我的好朋友们道一声感谢、送一句祝福。

首先，我要感谢我的校内导师杨菲老师，本文是在杨老师的精心指导下完成的，正是由于老师在论文上给我的意见与建议，才使我能够顺利的完成毕业论文的撰写，在这里再一次表达内心最真诚的感谢！与此同时，杨老师对学术认真的态度与对教育奉献的精神，也深深的影响与激励着我。接着，我要感谢培养单位的校外导师颜世云技术总监。在本文的选题、论证与写作方面，颜世云总监在自身工作即为繁忙的情况下依然抽出宝贵的时间给出我解疑答惑，多次与我深入探讨解决论文中出现的问题。在颜总监的身上我看到了国内科研工作者为国产高性能处理器事业的无怨无悔的付出与奉献，这也激励着我毕业后继续为国产集成电路事业奋斗的决心。

其次，我要感谢读研期间帮助过我的老师们，是老师们的辛勤付出，才为我们创造出了优良的学习环境。感谢培养单位的领导们，是他们的悉心帮助，才更好的让我融入了工作氛围中。感谢培养单位的陶涛、陆信良师兄，他们在我很多不懂的问题上都给了我无私的帮助与耐心的讲解。感谢培养单位的杜鑫师兄，杜师兄在本文的设计与数据分析方面也给过我很大的帮助。感谢三年研究生期间一起度过的同学们、朋友们，你们的相随也让偶尔枯燥的研究生生活充满欢声笑语。

最后，我要感谢我的父母家人们，是你们无怨无悔的付出，才让我能够有机会能够走入研究生的殿堂。你们的支持，一直是我前进的动力；你们的陪伴，永远是我坚实的后盾！